Graphics Output Primitives and their Implementation Algorithm

Graphics Output Primitives

1 Coordinate Reference Frames
2 Specifying a Two-Dimensional World-Coordinate Reference Frame in OpenGL
3 OpenGL Point Functions
4 OpenGL Line Functions
5 OpenGL Curve Functions
6 Fill-Area Primitives
7 Polygon Fill Areas
8 OpenGL Polygon Fill-Area Functions
9 OpenGL Vertex Arrays
10 Pixel-Array Primitives
11 OpenGL Pixel-Array Functions
12 Character Primitives
13 OpenGL Character Functions
14 Picture Partitioning
15 OpenGL Display Lists
16 OpenGL Display-Window Reshape Function
17 Summary



A general software package for graphics applications, sometimes referred to as a computer-graphics application programming interface (CG API), provides a library of functions that we can use within a programming language such as C++ to create pictures. The set of library functions can be subdivided into several categories. One of the first things we need to do when creating a picture is to describe the component parts of the scene to be displayed.

Implementation Algorithms for Graphics Primitives and Attributes

1 Line-Drawing Algorithms
2 Parallel Line Algorithms
3 Setting Frame-Buffer Values
4 Circle-Generating Algorithms
5 Ellipse-Generating Algorithms
6 Other Curves
7 Parallel Curve Algorithms
8 Pixel Addressing and Object Geometry
9 Attribute Implementations for Straight-Line Segments and Curves
10 General Scan-Line Polygon-Fill Algorithm
11 Scan-Line Fill of Convex Polygons
12 Scan-Line Fill for Regions with Curved Boundaries
13 Fill Methods for Areas with Irregular Boundaries
14 Implementation Methods for Fill Styles
15 Implementation Methods for Antialiasing
16 Summary



45

In this chapter, we discuss the device-level algorithms for implementing OpenGL primitives. Exploring the implementation algorithms for a graphics library will give us valuable insight into the capabilities of these packages. It will also provide us with an understanding of how the functions work, perhaps how they could be improved, and how we might implement graphics routines ourselves for some special application. Research in computer graphics is continually discovering new and improved implementation techniques to provide us with methods for special applications, such as Internet graphics, and for developing faster and more realistic graphics displays in general.

CHAPTER 3
&
CHAPTER 5

Outline

- Specifying 2D world-coordinate reference frame in OpenGL.
- OpenGL point functions.
- OpenGL line functions.
- Line-drawing algorithms (DDA, Bresenham's).
- Circle-generating algorithms (Bresenham's, midpoint).
- Midpoint Ellipse-generating algorithm.
- Polygon fill areas
- OpenGL polygon fill-area primitives.
- OpenGL vertex arrays.
- Other OpenGL functions (pixel-array, character,...).

- **Graphics Output Primitives:** Functions in a graphics package that we use to describe the various **picture components** are called the **graphics output primitives**, or simply **primitives.**
- **Picture components** could be trees and terrain, furniture and walls, storefronts and street scenes, automobiles and billboards, atoms and molecules, or stars and galaxies.
- The output primitives describing the geometry of objects are typically referred to as **geometric primitives**.

Basic output primitives

- OpenGL™ is a cross platform (hardware independent) 3D graphics library that takes advantage of specialized graphics hardware.
- Software graphics package is computer graphics application programming interface (CG API).
- Graphics output primitives are functions used to represent various picture components.
- Basic primitives are points, straight lines, circles, conic sections, curves, surfaces, polygon an filling color areas.
- Use OpenGL™ to deal with these primitives.

OpenGL Basics - OpenGL on Microsoft Windows

- **OpenGL API divided into 3 libraries**:
  - *1. OpenGL™ - The basic library*
  - *2. Glu - OpenGL Utility Library*
  - *3. Glut- A window toolkit*
- The basic OpenGL library and glu come bundled as part of Microsoft Windows. The libraries are stored as .lib files and as .dll files.

OpenGL Basics - OpenGL on Microsoft Windows

- The files and their locations are detailed in the following table (Next slide).
- The glut library (glut32.dll, glut32.lib, and glut.h) is not part of the standard Windows, but can be downloaded for free. (Visit site http://www.opengl.org/)
- See Table below : Use your compiler Visual C++ directory (as c:\Program Files\Microsoft Visual Studio\VC98).
- See Table below : Use your Windows 9x/NT/2000/XP system directory (such as c:\winnt\system32 or c:\windows\system)

OpenGL™ Basics - Install OpenGL on Microsoft Windows™

Library Name	Library File names	Header File (include)	Function Prefix
OpenGL (gl)	opengl32.lib opengl32.dll	gl.h	gl
Utility Library (glu)	glu32.lib glu32.dll	glu.h	glu
Toolkit Library (glut)	glut32.lib glut32.dll	glut.h	glut
Location	[compiler]\lib or [System] (*)	[compiler]\include\GL (*)	

Basic OpenGL Syntax

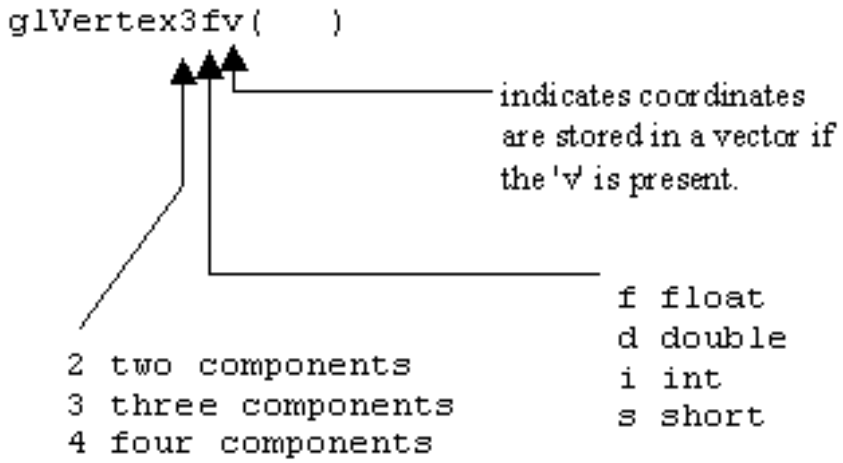
- Function names in the OpenGL basic library (also called the OpenGL core library) are prefixed with gl, e.g. glBegin, glClear, glCopyPixels, glPolygonMode
- Specifying symbolic constant in the OpenGL basic library e.g. GL_2D, GL_RGB, GL_CCW, GL_POLYGON, GL_AMBIENT_AND_DIFFUSE
- Specifying data types in the OpenGL basic library e.g. GLbyte, GLshort, GLint, GLfloat, GLdouble, Glboolean

OpenGL™ Output primitives

- Read Section 2.9 p.73 H&B CGwOGL
- Install Open GL™ on your system.
- Download *Glut* library from internet and install it in your *Microsoft Visual C++™ Include folder* as appropriate (Section 2.9 and Table above).
- For Open GL™ instructions relative to output primitives, see table 3.1 p. 158-159 for reference as and when needed.

Specifying Object Vertices

How to form a vertex call?



Every object is specified by vertices

- ***glVertex3f (2.0, 4.1, 6.0);***

// specifies a vertex at the x, y, z coordinate (2.0, 4.1, 6.0).

// The "3f" means 3 floating point coordinates.

- ***Other examples:***
  glVertex2i (4, 5);

// 2i means integers for x and y. z = 0.
glVertex3fv (vector);

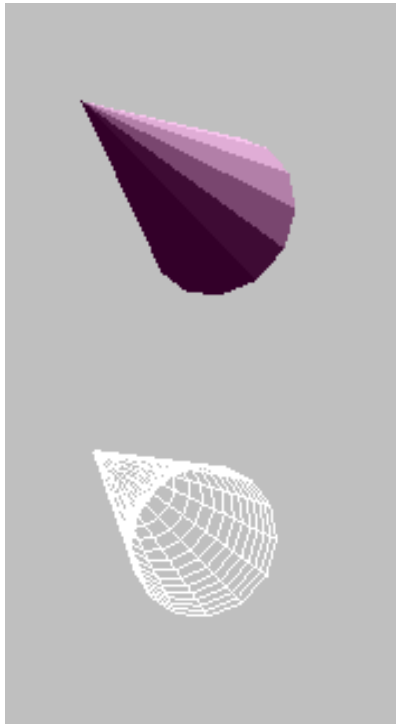
// float vector[3] = {5.0, 3.2, 5.0};

Primitive Types in glBegin

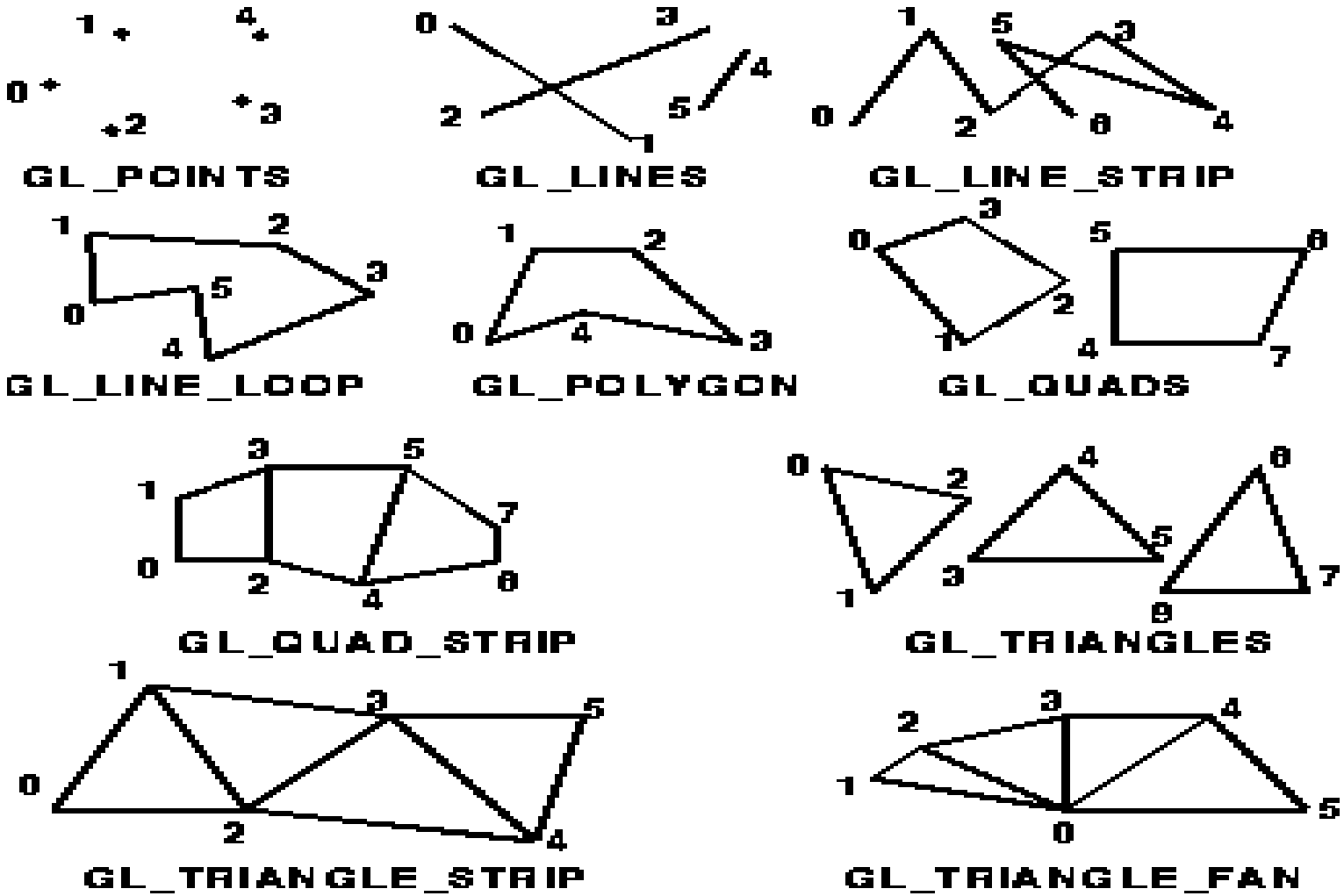
- **Points** GL_POINTS
- **Lines** GL_LINES, GL_LINE_STRIP, GL_LINE_LOOP
- **Triangles** GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN
- **Quads** GL_QUADS, GL_QUAD_STRIP
- **Polygons** GL_POLYGON

```
glBegin(GL_LINES);
   [lots of glVertex calls];
glEnd();

glBegin(GL_QUADS);
   [lots of glVertex calls];
glEnd();
```



Primitive Types in glBegin



Specifying Object Vertices

- Current color affects any vertices

  Example: ***glColor3f (0.0, 0.5, 1.0);***

  // no Red, half-intensity Green, full-intensity Blue

- Vertices are specified only between **glBegin(*mode*)** and **glEnd()**, usually in a counter-clockwise order for polygons.

```
Example: glBegin (GL_TRIANGLES);
        glVertex2i (0, 0);
        glVertex2i (2, 0);
        glVertex2i (1, 1);
glEnd();
```

Example of OpenGL™ program

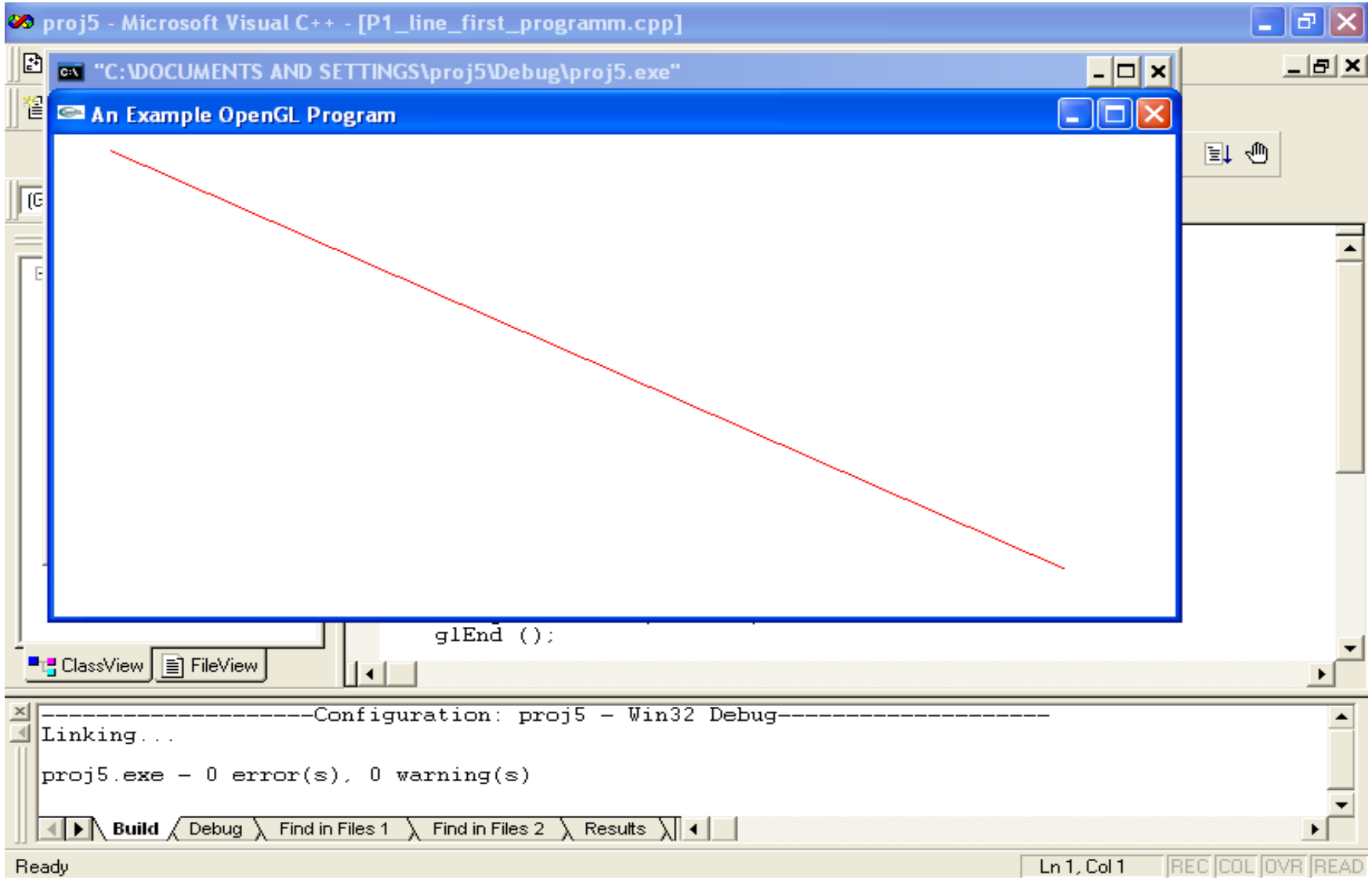
```
#include <GL/glut.h>      // (or others, depending on the system in use)
void init (void)
{    glClearColor (1.0, 1.0, 1.0, 0.0);  // Set display-window color to white, 4th
param. is alpha – Opacity- transpancy level.

    glMatrixMode (GL_PROJECTION);      // Set projection parameters.
    gluOrtho2D (0.0, 200.0, 0.0, 150.0); // orthogonal projection, in 2 dimension, x
& y
}
void lineSegment (void)
{
    glClear (GL_COLOR_BUFFER_BIT);  // Clear display window, To get the assigned
window color displayed
    glColor3f (1.0, 0.0, 0.0);      // Set line segment color to red.
    glBegin (GL_LINES);
        glVertex2i (180, 15);      // Specify line-segment geometry.
        glVertex2i (10, 145);
    glEnd ( );
    glFlush ( );     // Process all OpenGL routines as quickly as possible.
}
```

Example of OpenGL™ program (Cont.)

```
void main (int argc, char** argv)
{   glutInit (&argc, argv);                          // Initialize GLUT.
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);   // Set display mode, single
refresh buffer & RGB.
    glutInitWindowPosition (50, 100);   // Set top-left display-window position.
    glutInitWindowSize (400, 300);      // Set display-window width and height.
    glutCreateWindow ("An Example OpenGL Program"); // Create display window.
    init ( );                           // Execute initialization procedure.
    glutDisplayFunc (lineSegment);      // Send graphics to display window.
    glutMainLoop ( );                   // Display everything and wait.
}
```

Example of OpenGL™ program



Coordinate reference frame

- To describe a picture - convenient Cartesian coordinate system, called the world-coordinate reference frame – 2D or 3D.
- Then describe the objects in our picture -by giving their geometric specifications in terms of positions in world coordinates. For instance, we define a straight-line segment with two end point positions ,and a polygon is specified with a set of positions for its vertices. These coordinate positions are stored in the scene description along with other information about the objects, such as their color and their **coordinate extents.**

Screen Coordinates

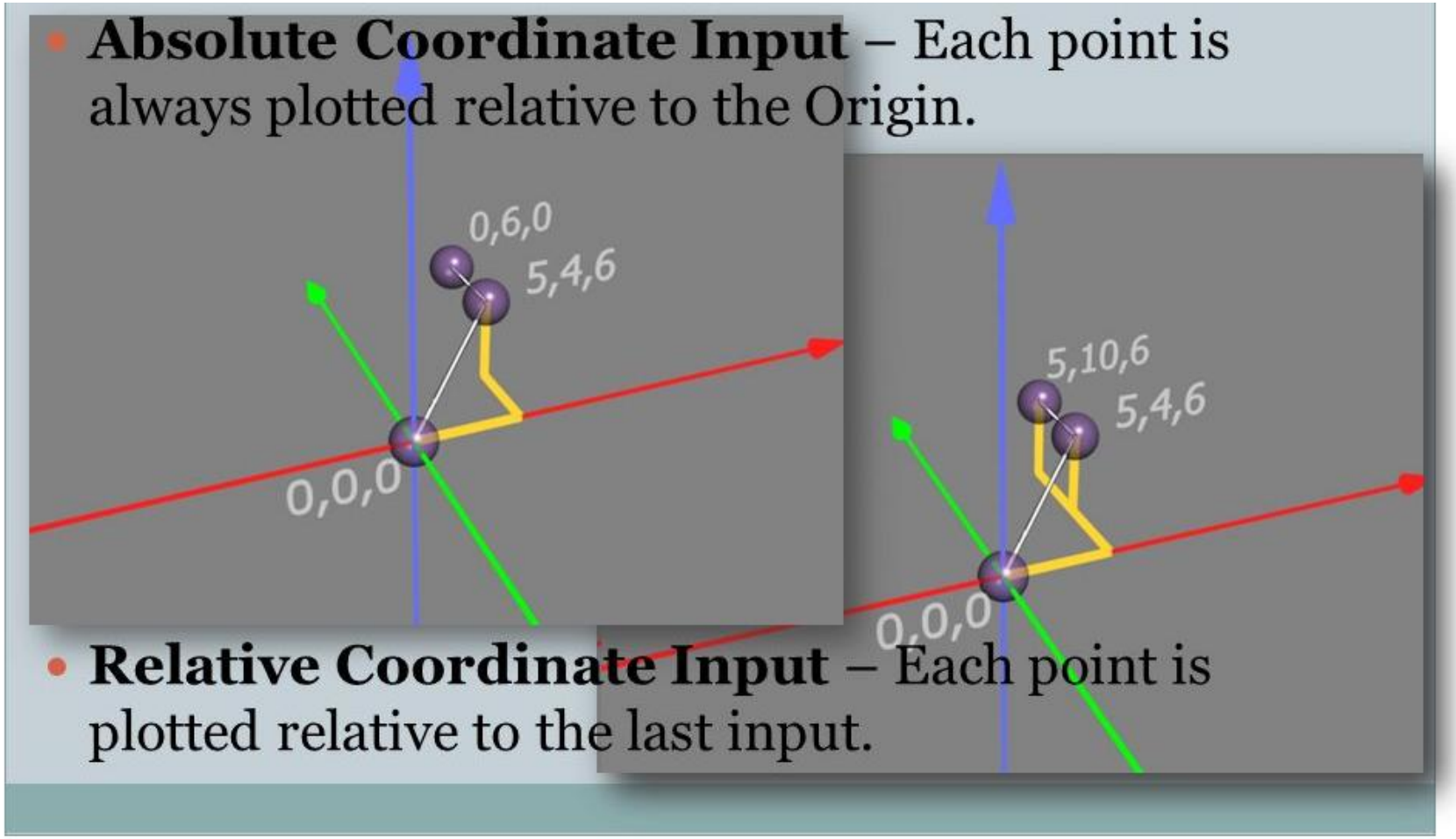
- Locations on a video monitor are referenced in integer screen coordinates, which correspond to the pixel positions in the frame buffer. Pixel coordinate values give the scan line number (the y value) and the column number (the x value along a scan line). Hardware processes, such as screen refreshing, typically address pixel positions with respect to the top-left corner of the screen. Scan lines are then referenced from 0, at the top of the screen, to some integer value, y max, at the bottom of the screen, and pixel positions along each scan line are numbered from 0 to x max, left to right. However, with software commands, we can setup any convenient reference frame for screen positions.

Screen Coordinates

- Screen coordinates are pixel positions in frame buffer.
- In OpenGL™, low-level functions:
- **1. setPixel(x,y)** stores current color setting into frame buffer at integer position (x, y).
- **2. getPixel(x,y,color)** retrieves current frame buffer setting for a pixel location (x, y)
- Although, we need only specify color values at (x, y) positions for a two dimensional picture, additional screen-coordinate information is needed for three-dimensional scenes. In this case, screen coordinates are stored as three dimensional values, where the third dimension references the depth of object positions relative to a viewing position. For a two-dimensional scene, all depth values are 0.

Absolute and Relative Coordinates

- So far, the coordinate references that we have discussed are stated as absolute coordinate values. This means that the values specified are the actual positions within the coordinate system in use.
- However, some graphics packages also allow positions to be specified using relative coordinates.
- In this approach, we can specify a coordinate position as an offset from the last position that was referenced (called the **current position**).
- **E.g.** Current position=(3, 8), a relative coordinate specification of (2, −1) –> absolute position of (5, 7).
- To describe an object, such as a series of connected line segments, Star
- We will mostly use absolute coordinates.


Absolute Coordinate Input – Each point is always plotted relative to the Origin.
0,6,0
5,4,6
0,0,0
5,10,6
5,4,6
0,0,0
Relative Coordinate Input – Each point is plotted relative to the last input.

Specifying A Two-Dimensional World-Coordinate Reference Frame In OpenGL

- ***gluOrtho2D*** - a function we can use to setup any two-dimensional Cartesian reference frame.
- Arguments for this function are the four values defining the x and y coordinate limits for the picture we want to display.
- For initial two-dimensional examples, we can define the coordinate frame for the screen display window with the statements in next slide.
- The display window will then be referenced by coordinates (xmin, ymin) at the lower-left corner and by coordinates (xmax, ymax) at the upper-right corner

2D World-coordinate reference frame

- Create an orthogonal 2D reference frame with (x_{min}, y_{min}) at the low-level corner and (x_{max}, y_{max}) at the upper-right corner

- **glMatrixMode (GL_PROJECTION);** // Set projection parameters.
  glLoadIdentity (); //(re-)initialize coordinates
  gluOrtho2D (x_{min}, x_{max}, y_{min} , y_{max});

2D World-coordinate reference frame

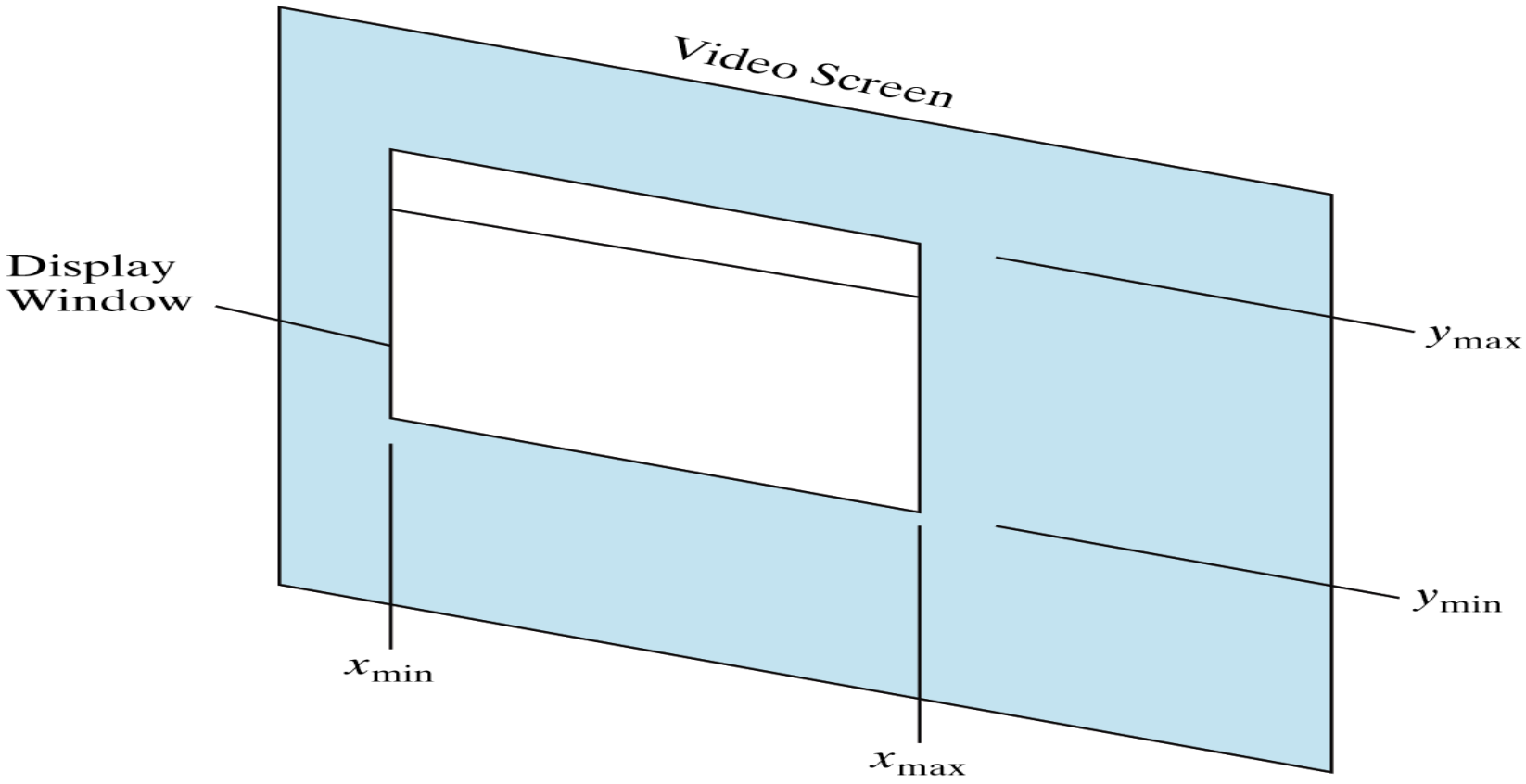


Figure 3-2

World-coordinate limits for a display window, as specified in the glOrtho2D function.

- We can now designate one or more graphics primitives for display using the coordinate reference specified in the gluOrtho2D statement
- If the coordinate extents of a primitive are within the coordinate range of the display window, all of the primitive will be displayed.
- Otherwise, only those parts of the primitive within the display-window coordinate limits will be shown.
- When we setup the geometry describing a picture, all positions for the OpenGL primitives must be given in absolute coordinates, with respect to the reference frame defined in the gluOrtho2D function.

OpenGL™ point functions

- To specify the geometry of a point, we simply give a coordinate position in the world reference frame.
- OpenGL primitives are displayed with a default size and color . The default color for primitives is white and the default point size is equal to the size of one screen pixel.
- OpenGL function to state the coordinate values for a single position

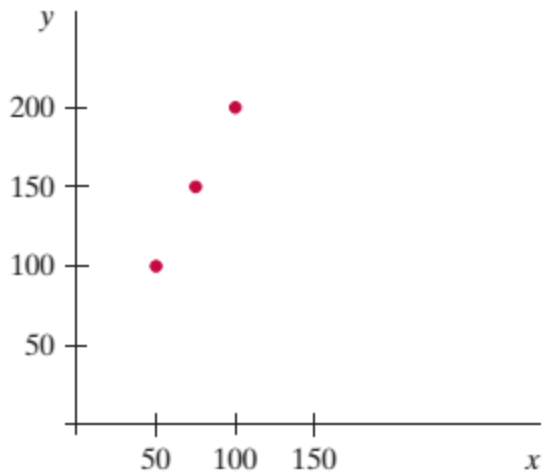
glVertex* ();

- where the asterisk(*) indicates that suffix codes are required for this function.

OpenGL™ point functions

- The argument of the glBegin function is used to identify the kind of output primitive that is to be displayed ,and glEnd takes no arguments.
- In OpenGL we can specify coordinate positions in 2,3, or 4 dimensions, the 4th dimension is used for *homogeneous coordinates* h (as scaling factor for the Cartesian-coordinate values).
- Example1 , draw 3 points on a straight line of slope 2.

```
glBegin (Gl_POINTS)
  glVertex2i(50,100)
  glVertex2i(75,150)
  glVertex2i(100,200)
glEnd();
```



OpenGL™ point functions

- we could specify the coordinate values for the preceding points in arrays such as

```
intpoint1 [] = {50, 100};
intpoint2 [] = {75, 150};
intpoint3 [] = {100, 200};
```

- and call the OpenGL functions for plotting the three points as

```
glBegin (GL_POINTS);
glVertex2iv(point1);
glVertex2iv(point2);
glVertex2iv(point3);
glEnd ();
```

- And here is an example of specifying two point positions in a three dimensional world reference frame. In this case, we give the coordinates as explicit floating-point values.

```
glBegin (GL_POINTS);
glVertex3f (-78.05, 909.72, 14.60);
glVertex3f (261.91, -5200.67, 188.33);
glEnd ();
```

- We could also define a C++ class or structure (struct) for specifying point positions in various dimensions. For example,

```
class wcPt2D {
public:
Glfloat x, y;
};
```

- Using this class definition, we could specify a two-dimensional, world-coordinate point position with the statements

```
wcPt2D pointPos;
pointPos.x =120.75;
pointPos.y =45.30;
glBegin (GL_POINTS);
glVertex2f (pointPos.x, pointPos.y);
glEnd ();
```

- **coordinate extents** - are the minimum and maximum x, y, and z values for each object.
- A set of coordinate extents is also described as a bounding box for an object.
- For a two-dimensional figure, the coordinate extents are sometimes called an object's bounding rectangle.
- Objects are then displayed by passing the scene information to the viewing routines, which identify visible surfaces and ultimately map the objects to positions on the video monitor.

OpenGL™ line functions

- The following code draws lines between p1 and p2 and line between p3 and p4 and ignores p5 (Fig 3.4a p. 91)

```
glBegin (Gl_LINES)//Or GL_LINE_STRIP) Or (GL_LINE_LOOP)
  glVertex2iv(p1)// iv integer vector.
  glVertex2iv(p2)
  glVertex2iv(p3)
  glVertex2iv(p4)
  glVertex2iv(p5)
```

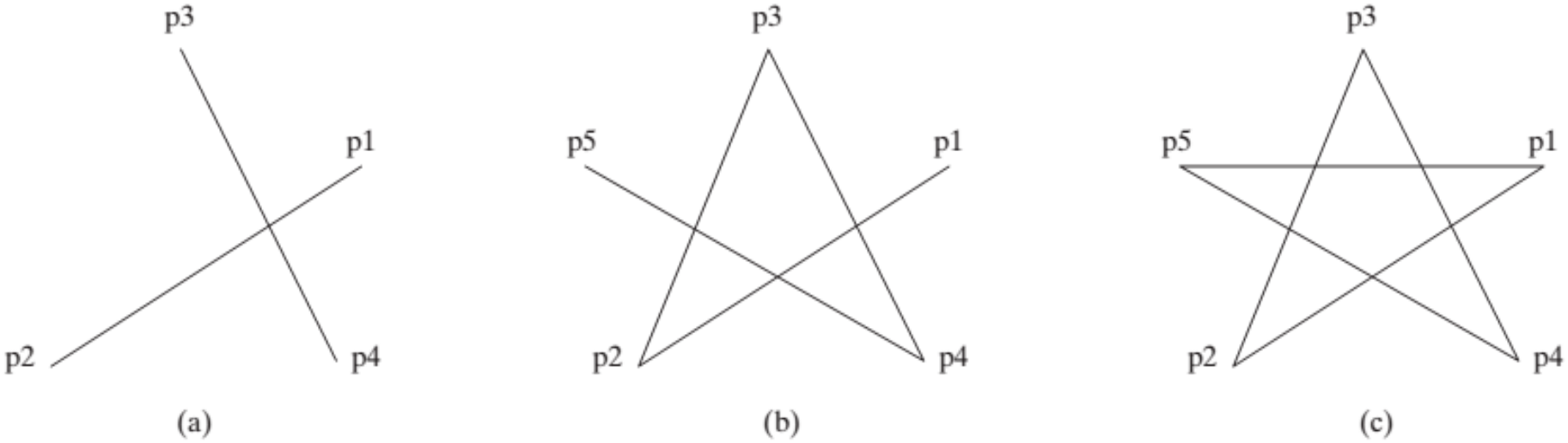


FIGURE 3-4 Line segments that can be displayed in OpenGL using a list of five endpoint coordinates. (a) An unconnected set of lines generated with the primitive line constant GL_LINES. (b) A polyline generated with GL_LINE_STRIP. (c) A closed polyline generated with GL_LINE_LOOP.

LINE-DRAWING

- To display the line on a raster monitor, the graphics system must first project the endpoints to integer screen coordinates and determine the nearest pixel positions along the line path between the two end points. Then the line color is loaded into the frame buffer at the corresponding pixel coordinates. Reading from the frame buffer, the video controller plots the screen pixels.
- Computed line position/s is/are rounded to integers causes all but horizontal and vertical lines to be displayed with a stair-step appearance:



FIGURE 3–5 Stair-step effect (jaggies) produced when a line is generated as a series of pixel positions.

Equation of a straight line

- The Cartesian slope-intercept equation for a straight line is:

1. $Y = m \cdot x + b$ Where m=slope of line, b= y-intercept
2. $m = (y_{end} - y_o)/(x_{end} - x_o)$ (ميل)
3. $b = y_o - m \cdot x_o$

- For any given x interval δ x along a line, we can compute the corresponding y interval δy from Eq. 2 as

4. $\delta y = m \cdot \delta x$

- Similarly , we can obtain the x interval δx corresponding to a specified δy as

5. $\delta x = \delta y / m$

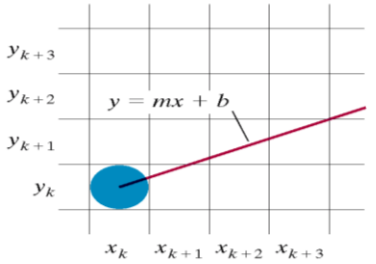


Figure 3-10

A section of the screen showing a pixel in column x_k on scan line y_k that is to be plotted along the path of a line segment with slope $0 < m < 1$.

- For lines with slope magnitudes $|m| < 1$, δx can be set proportional to a small horizontal deflection voltage, and the corresponding vertical deflection is then set proportional to δy as calculated from Eq. 4.
- For lines whose slopes have magnitudes $|m| > 1$, δy can be set proportional to a small vertical deflection voltage with the corresponding horizontal deflection voltage set proportional to δx, calculated from Eq. 5.
- For lines with $m = 1$, $\delta x = \delta y$ and the horizontal and vertical deflections voltages are equal.

Line-drawing algorithms DDA

- Direct Differential Algorithm (DDA)
- Consider both cases for positive and negative slopes.
- Floating increment coordinates calculations then rounded to integer
- Rounding operations of the floating point calculations cause pixel positions to drift away from the true line path for long line segments
- Rounding operations are time consuming.

Line-drawing algorithms
DDA : mathematical derivation

$$(x_k \ , y_k) \quad (1)$$

$$(x_{k+1} \ , y_{k+1}) \quad (2)$$

$$\frac{\Delta y}{\Delta x} = m = \frac{y_{k+1} - y_k}{x_{k+1} - x_k} \quad (3)$$

$$y_{k+1} = y_k + m\Delta x \quad (4)$$

$$x_{k+1} = x_k + \frac{\Delta y}{m} \quad (5)$$

Line-drawing algorithms
DDA : mathematical derivation

$$Case:1 \quad |m| \le 1 \quad choose \quad x \quad as \quad \text{variable}$$

$$Unit\ steps \quad \Delta x = 1$$

$$Deduce \quad y_{k+1} = y_k + m \quad (6)$$

$$Case:2 \quad |m| > 1 \quad choose \quad y \quad as \quad \text{variable}$$

$$Unit\ steps \quad \Delta y = 1$$

$$Deduce \quad x_{k+1} = x_k + 1/m \quad (7)$$

Line-drawing algorithms
DDA : mathematical derivation

- Choose two consecutive points (1) and (2)
- Define slope of straight line by (3).
- Based on the equations (4), (5), calculate coordinates.
- Based on equation(6) calculate coordinates for case 1.
- Based on equation(7) calculate coordinates for case 2.

Line-drawing algorithms - DDA source code

```
#include <stdlib.h>
#include <math.h>

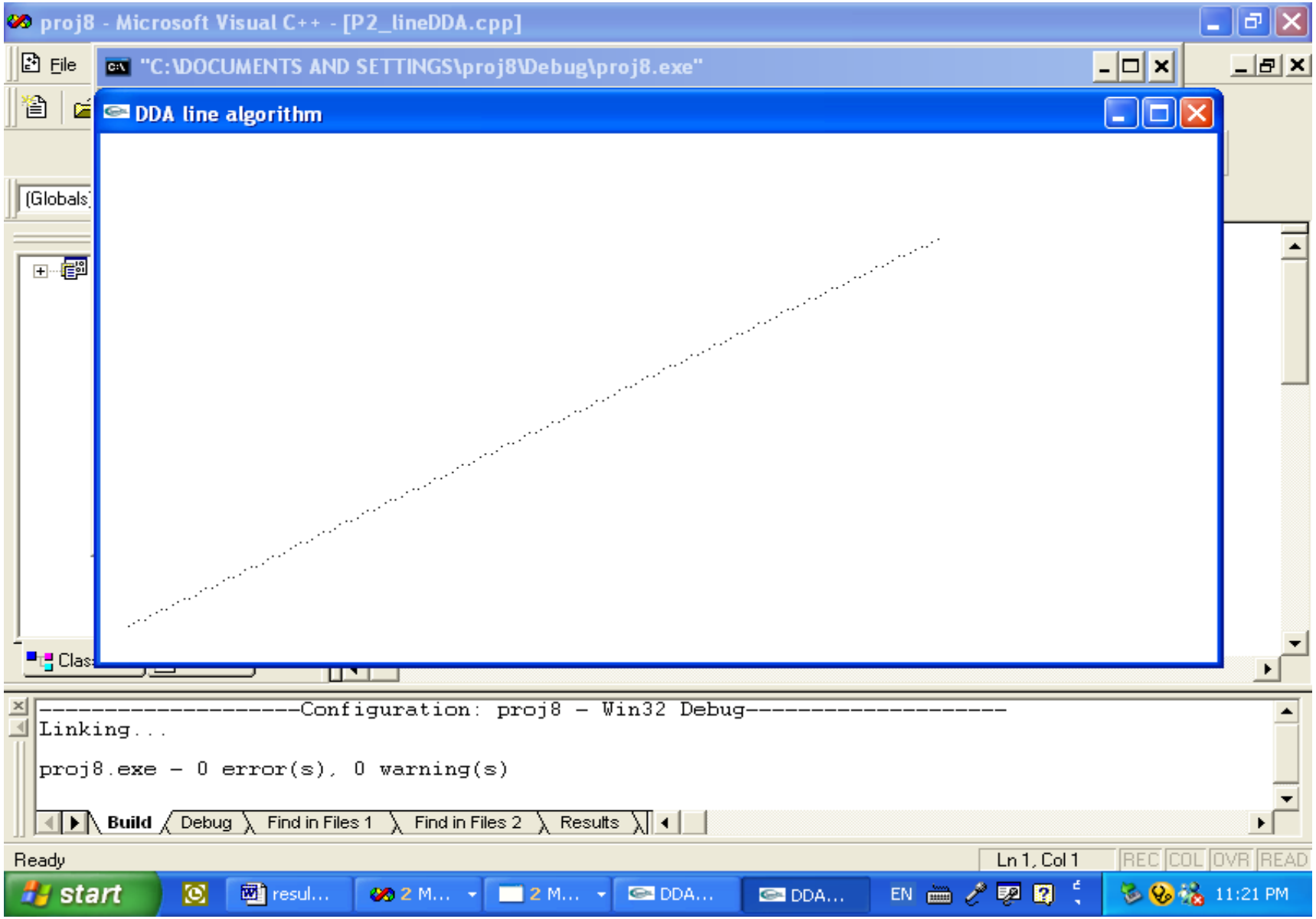
  inline int round (const float a)  { return int (a + 0.5); }

  void lineDDA (int xo, int yo, int xEnd, int yEnd)
  {
    int dx = xEnd - xo,  dy = yEnd - yo,  steps,  k;
    float xIncrement, yIncrement, x = xo, y = yo;

    if (fabs (dx) > fabs (dy))
      steps = fabs (dx);
    else
      steps = fabs (dy);
    xIncrement = float (dx) / float (steps);
    yIncrement = float (dy) / float (steps);

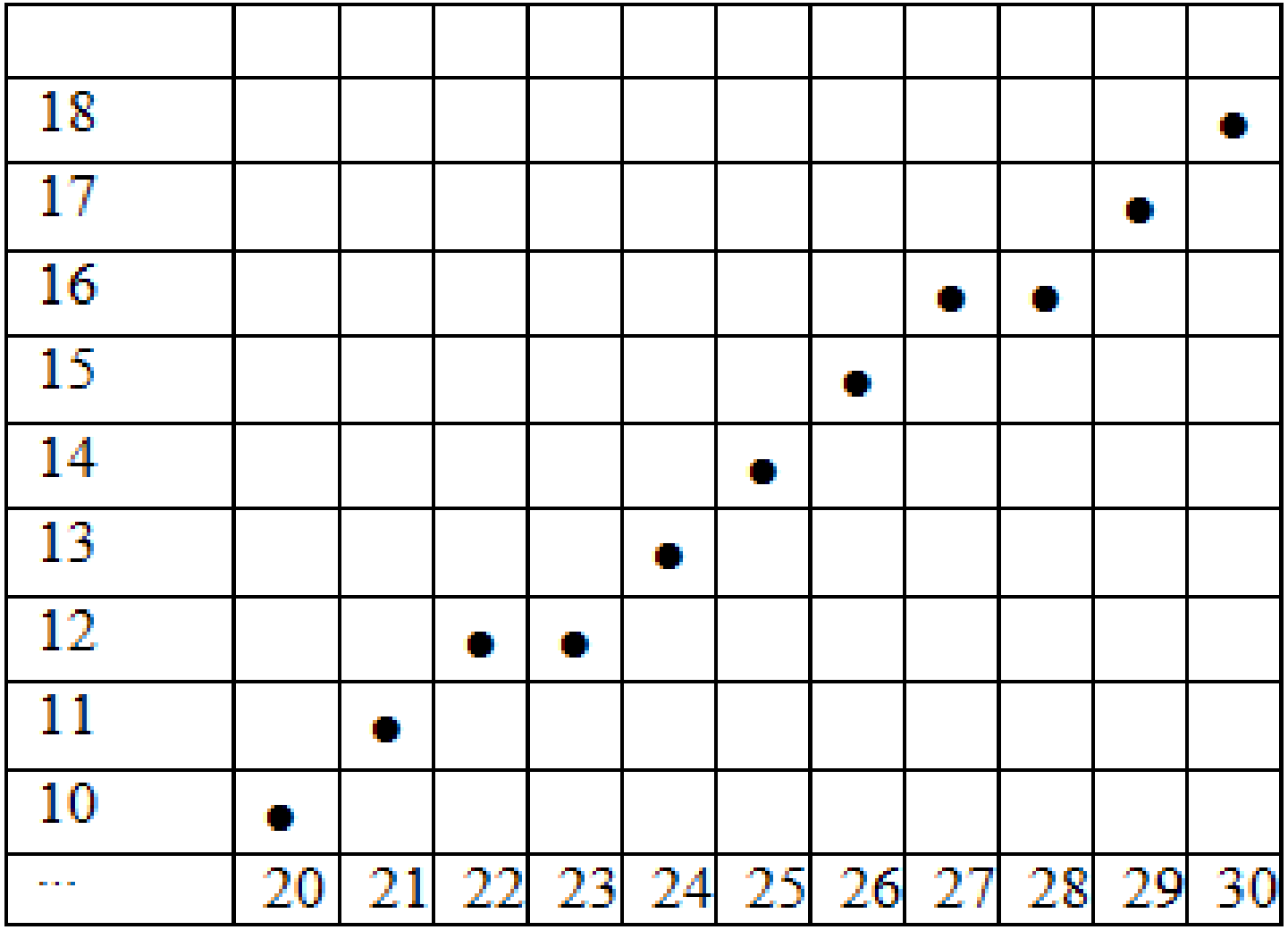
    setPixel (round (x), round (y));
    for (k = 0; k < steps; k++) {
      x += xIncrement;
      y += yIncrement;
      setPixel (round (x), round (y));
    }
```

Line-drawing algorithms - DDA test of code



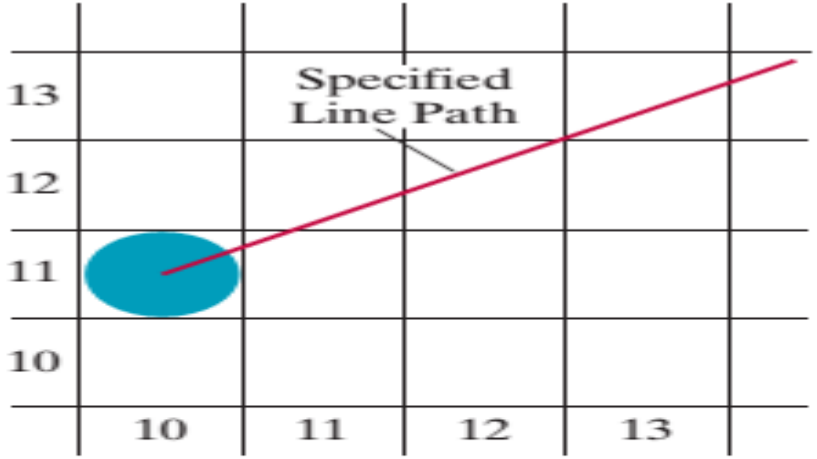
Line From (20,10) to (30,18), m= (18-10)/(30-20) = 8/10 = 0.8

X	Y	Round(y)
20	10	
21	y = y + m = 10 + 0.8=10.8	11
22	y = y + m = 10.8 + 0.8=11.6	12
23	y = y + m = 11.6 + 0.8=12.4	12
24	y = y + m = 12.6 + 0.8=13.2	13
25	y = y + m = 13.2 + 0.8=14.0	14
26	y = y + m = 14 + 0.8=14.8	15
27	y = y + m = 14.8 + 0.8=15.6	16
28	y = y + m = 15.6 + 0.8=16.4	16
29	y = y + m = 16.4 + 0.8=17.2	17
30	y = y + m = 17.2 + 0.8=18.0	18


18
17
16
15
14
13
12
11
10
...
20
21
22
23
24
25
26
27
28
29
30

Line-drawing algorithms
Bresenham's algorithm

- Consider both cases for positive and negative slopes.
- Accurate and efficient raster line-generating algorithm.
- More complex mathematical model
- More accurate uses only integer calculations
- Can be adapted to display circles and other curves



+ve slope & _ve Slope ->

FIGURE 3–8 A section of a display screen where a straight-line segment is to be plotted, starting from the pixel at column 10 on scan line 11.

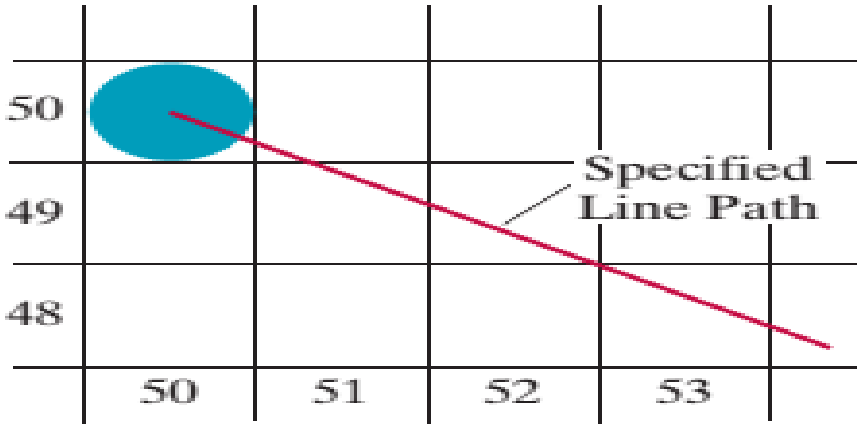


FIGURE 3–9 A section of a display screen where a negative slope line segment is to be plotted, starting from the pixel at column 50 on scan line 50.

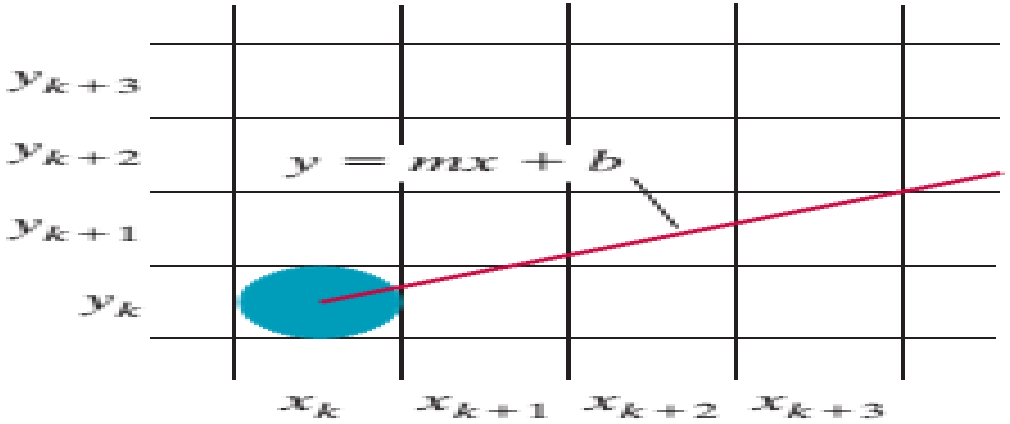


FIGURE 3–10 A section of the screen showing a pixel in column x_k on scan line y_k that is to be plotted along the path of a line segment with slope $0 < m < 1$.

- Bresenham's approach: +ve slope m <1.0. Pixel positions along a line path are then determined by sampling at unit x intervals. Start from the left endpoint (xo, yo). we step to each successive column (x position) and plot the pixel whose scan-line y value is closest to the line path. pixel at (x_k, y_k) is displayed, we next need to decide which pixel to plot in column $x_{k+1} = x_k + 1$. Our choices are the pixels at positions (x_{k+1}, y_k) and (x_{k+1}, y_{k+1}).

Line-drawing algorithms - Bresenham's algorithm

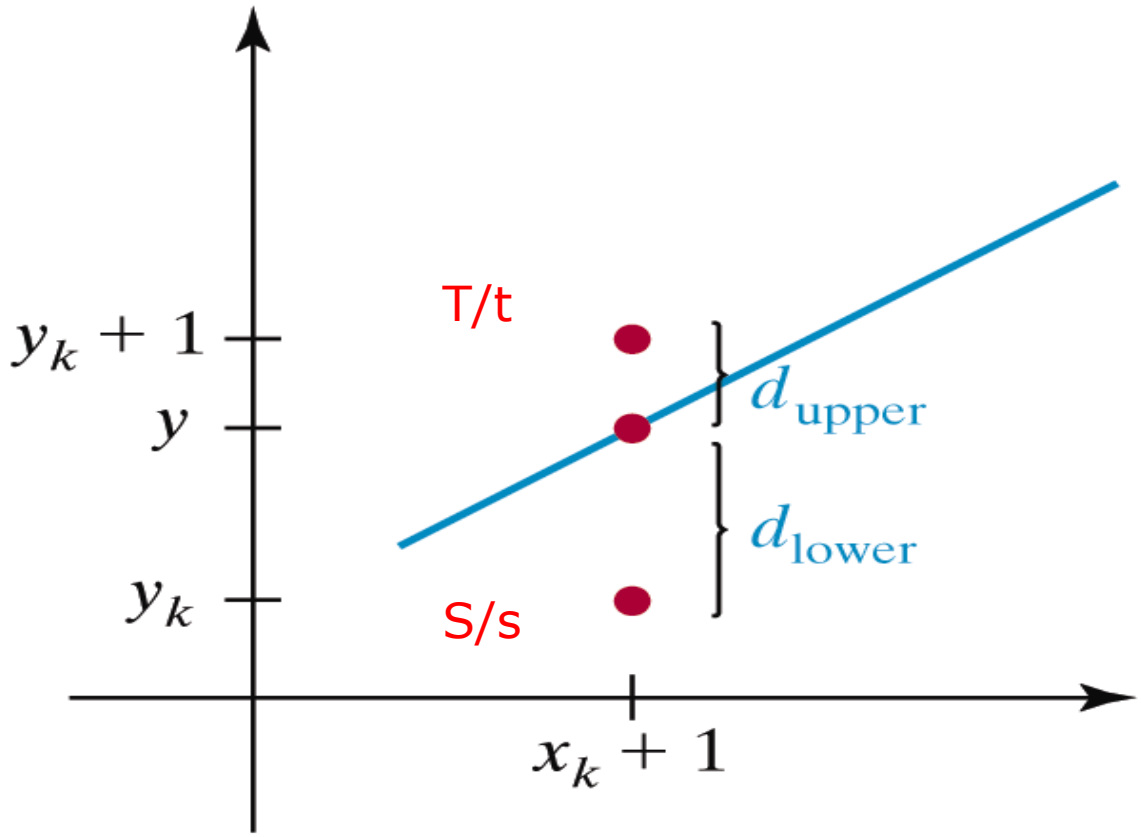
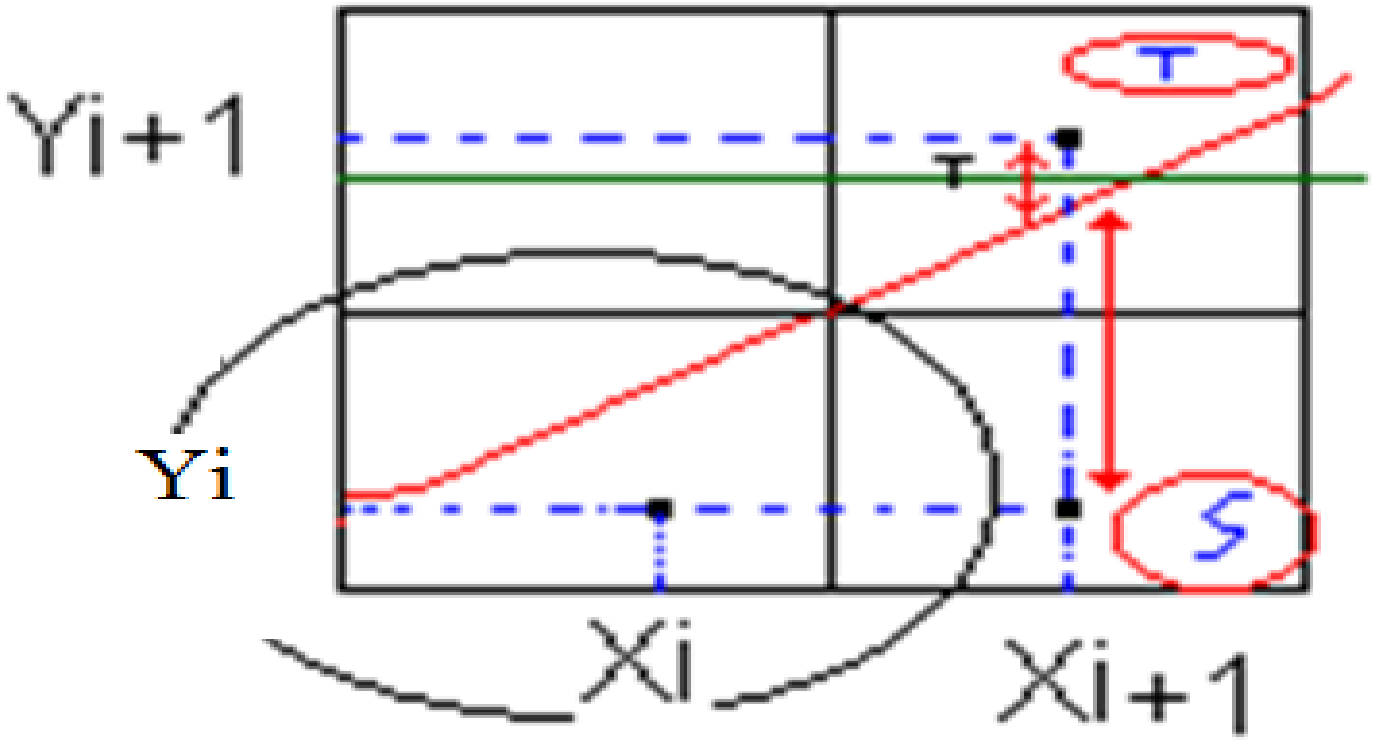


Figure 3-11

Vertical distances between pixel positions and the line y coordinate at sampling position $x_k + 1$.


Yi+1
T
T
Yi
S
Xi
Xi+1

Line-drawing algorithms
Bresenham's algorithm

- Testing the sign of an integer parameter whose value proportional to difference between the vertical separations of the two pixels from the true line path (sampling at unit X interval)
- Starting from pixel (x_0,y_0) we plot the nearest pixel to line path with

Line-drawing algorithms - Bresenham's algorithm - mathematics

- Start at (x_i , y_i) Consider previous figures 40 & 41.

$0 < m < 1$ $\quad (x_i , y_i)$

$T(x_i + 1, y_i + 1)$ $\quad S(x_i + 1, y_i)$

$y = m.x + b$

$$y = mx_{i+1} + b \quad (1)$$

$$y = m(x_i + 1) + b \quad (2)$$

d_{lower} $\quad s = y - y_i \quad (3)$

d_{upper} $\quad t = y_{i+1} - y \quad (4)$

The y coordinate on the mathematical line at pixel column position $x_k + 1$ is calculated as

- Consider a variant of the decision variable based on the difference

$$s - t = (y - y_i) - [(y_i + 1) - y] \quad (5)$$

$$= y - y_i - y_i - 1 + y$$

$$s < t \Rightarrow (s - t) < 0$$

$$s > t \Rightarrow (s - t) > 0$$

$$s = t \Rightarrow s - t = 0$$

$$s - t = 2y - 2y_i - 1 \quad (6)$$

Line-drawing algorithms
Bresenham's algorithm
mathematics…

The difference becomes

$$s - t = 2y - 2y_i - 1$$

$$= 2m(x_i + 1) + 2b - 2y_i - 1$$

$$s - t = 2mx_i + 2m + 2b - 2y_i - 1$$

$$m = \frac{\Delta y}{\Delta x} \qquad (10)$$

$$\therefore s - t = 2\frac{\Delta y}{\Delta x} \cdot x_i + 2\frac{\Delta y}{\Delta x} + 2b - 2y_i - 1 \qquad (11)$$

To determine which of the two pixels is closest to the line path, we setup an efficient test that is based on the difference between the two pixel separations

- Consider now the decision variable with same sign as the previous: (P_k is decision parameter)

$$p_k = \Delta x(s - t) \qquad (12)$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.2 Second : … mathematics…

- The decision variable P_k becomes:

$$p_k = \Delta x\left[2\frac{\Delta y}{\Delta x}x_k + 2\frac{\Delta y}{\Delta x} + 2b - 2y_k - 1\right] \qquad (13)$$

$$= 2\Delta y.x_k + 2\Delta y + 2b\Delta x - 2\Delta x.y_k - \Delta x \cdot$$

$$= 2\Delta y.x_k - 2\Delta x \cdot y_k + 2\Delta y + 2b\Delta x - \Delta x$$

$$= 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + 2\Delta y + \Delta x(2b-1) \cdot \qquad (14)$$

$$\underbrace{\qquad\qquad\qquad}_{C}$$

$$p_k = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c \qquad (15)$$

- Parameter c is constant and has the value $2\Delta y + \Delta x(2b - 1)$, which is independent of the pixel position and will be eliminated in the recursive calculations for p_k

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.2 Second : … mathematics…

The difference between 2 consecutive decision variables is Knowing that d_{i+1} is :

$$p_{k+1} = \Delta y \cdot x_{k+1} - 2\Delta x y_{k+1} + c \qquad (16)$$

$$p_{k+1} - p_k = 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k) \qquad (17)$$

$$x_{k+1} - x_k = 1 \qquad (18)$$

$$p_{k+1} - p_k = 2\Delta y - 2\Delta x(y_{k+1} - y_k) \qquad (19)$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.2 Second : … mathematics…

- There are 2 cases. One of them is:

$$case \quad 1$$

$$p_k \geq 0 \Rightarrow s - t \geq 0 \quad choice \quad of \quad pixel \quad T$$

$$y_{k+1} = y_k + 1 \qquad (20)$$

$$p_{k+1} - p_k = 2\Delta y - 2\Delta x \qquad (21) \qquad \therefore$$

$$p_{k+1} = p_k + 2(\Delta y - \Delta x) \quad , \; p_k \geq 0 \qquad (22)$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.2 Second : … mathematics…

- The other case is:

$$case \quad 2$$

$$p_k < 0 \Rightarrow s - t < 0 \quad choice \quad of \quad pixel \quad S$$

$$p_{k+1} - p_k = 2\Delta y \qquad y_{k+1} = y_k$$

$$p_{k+1} = p_k + 2\Delta y \qquad p_k < 0 \qquad (23)$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.2 Second : … mathematics…

- At last the final result is (nearly) found…
- But…

$$p_{k+1} = \begin{cases} p_k + 2(\Delta y - \Delta x) & p_k \geq 0 \\ p_k + 2\Delta y & p_k < 0 \end{cases} \qquad (24)$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.2 Second : … mathematics…

- … But we need the initial value of the decision variable:

Calculate initial value

$$p_0 = \Delta x\left[2\frac{\Delta y}{\Delta x}(x_o + 1) + 2b - 2y_0 - 1\right] \qquad (25)$$

$$= \Delta x[2(mx_0 + b - y_0) + 2m - 1]$$

$$(x_0, y_0)\ \text{initial point} \Rightarrow y_0 = mx_0 + b$$

$$\Rightarrow mx_0 + b - y_0 = 0$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm - 6.2 Second : … mathematics…

- Calculate p_o & p_1: The first parameter, p_o, is evaluated from Eq. 15 at the starting pixel position (x_o, y_o) and with m evaluated as $\Delta y/\Delta x$:

$$P_0 = \Delta x[2m-1] \quad (26)$$

$$= \Delta x\left[\frac{2\Delta y}{\Delta x} - 1\right] \quad (27)$$

$$p_0 = [2\Delta y - \Delta x]$$

$$p_1 = p_0 + 2(\Delta y - \Delta x)$$

$$= 4\Delta y - 3\Delta x$$

$$p_0 \geq 0 \quad (28)$$

$$p_1 = p_0 + 2\Delta y$$

$$= 4\Delta y - \Delta x$$

$$p_0 < 0 \quad (29)$$

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.3...Third : ...pseudo code...

1. Input the 2 line endpoints and store the left endpoint in $(x0, y0)$
2. Select color for frame-buffer position $(x0, y0)$ i.e. plot first point
3. Calculate the constants Δx, Δy, $2\Delta y$, $2\Delta y - 2\Delta x$ and get starting value of decision parameter do= $2\Delta y - \Delta x$
4. At each xk along the line, starting at i=0, perform the following test:
   (a)Pk<0 plot (xk+1, yk); pk+1=pk+2Δy
   (b)Pk>=0, plot (xi+1, yi+1); pk+1=pk+2Δy−2Δx
5. Repeat step 4, $\Delta x - 1$ times.

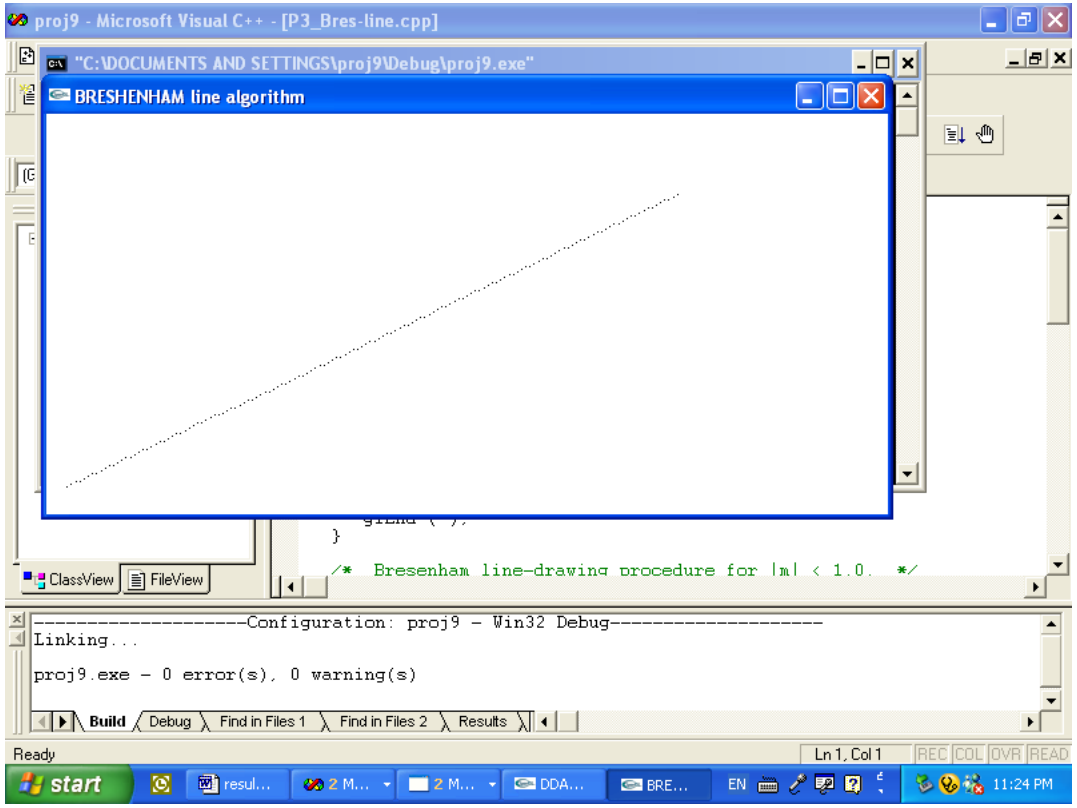
4.5 Line-drawing algorithms
6. Bresenham's algorithm - 6.4 …Fourth : …source code (adapt it!)

```
#include <stdlib.h>
 #include <math.h>    /*  Bresenham line-drawing procedure for |m| < 1.0.  */
  void lineBres (int xo, int yo, int xEnd, int yEnd)
  {    int dx = fabs (xEnd - xo),  dy = fabs(yEnd - yo);
    int d = 2 * dy - dx;
    int twoDy = 2 * dy,  twoDyMinusDx = 2 * (dy - dx);
    int x, y;
    /* Determine which endpoint to use as start position.  */
    if (xo > xEnd) {
      x = xEnd;
      y = yEnd;
      xEnd = xo;       }
    else {   x = xo;
             y = yo;              }
    setPixel (x, y);
```

4.5 Line-drawing algorithms
6. Bresenham's algorithm - 6.4 …Fourth : …source code (adapt it!)

```
while (x < xEnd)
        {  x++;
           If (d < o)
           d+ = twoDy;
           else
            {       y++;
                    d += twoDyMinusDx;    }
      setPixel (x, y);
   }  /* end while */        }  /*lineBres */
```

4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.5 Last but not least : ...testing source code



4.5 Line-drawing algorithms
6. Bresenham's algorithm
6.5 Last but not least : …testing source code

- Test previous code during labs sessions and make changes you think are important and/or interesting.
- Get more hands-on experience on similar problems.
- Use both Textbook and the Red Book.
- Get more experience for your project!
- No true swimming without use of a true swimming pool (or equivalent).

And…

- For a line with positive slope greater than 1.0, we interchange the roles of the x and y directions. That is, we step along the y direction in unit steps and calculate successive x values nearest the line path.
- Also, we could revise the program to plot pixels starting from either endpoint. If the initial position for a line with positive slope is the right end point, both x and y decrease as we step from right to left.
- We always chose the upper (or the lower) of the two candidate pixels whenever the vertical separations from the line path are equal ($d_{lower} = d_{upper}$).
- For negative slopes, the procedures are similar, except that now one coordinate decreases as the other increases.
- Finally, special cases can be handled separately: Horizontal lines ($\Delta y = 0$), vertical lines ($\Delta x = 0$),and diagonal lines ($|\Delta x|=|\Delta y|$) can each be loaded directly into the frame buffer without processing them through the line-plotting algorithm.

Ex: Find the co-ordinates that will be plotted using Bresenham's Line Drawing Algorithm, when the input co-ordinates are: (20, 10) and (30, 18) ??

no iterations	di	x	y

4.6 OpenGL curve functions

- Application of line-drawing algorithms
- Approximate an arc with 3, 6 and 12 vertexes

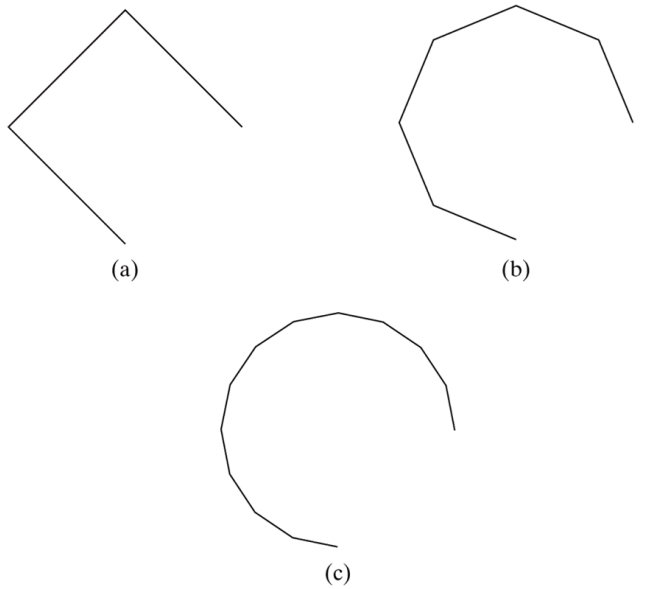


Figure 3-15

A circular arc approximated with (a) three straight-line segments, (b) six line segments, and (c) twelve line segments.

4.7 Circle-generating algorithms
1. Introduction

- Read p. 103-109 Textbook.
- See detailed complementary mathematical derivations of Bresenham's algorithm done in class
- Refer to fig. 3.18 below for reducing number of points to scan (symmetry).
- Exercise: use similar ideas to derive the midpoint circle algorithm (see Fig. 3.19 below).

Properties of Circle

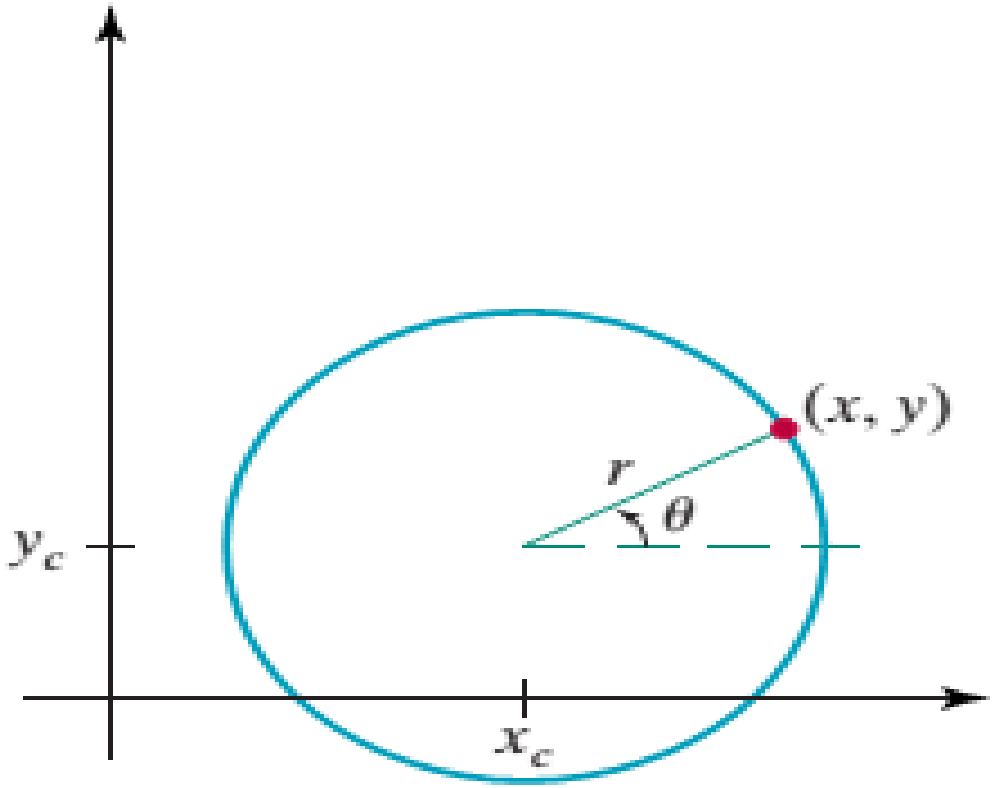
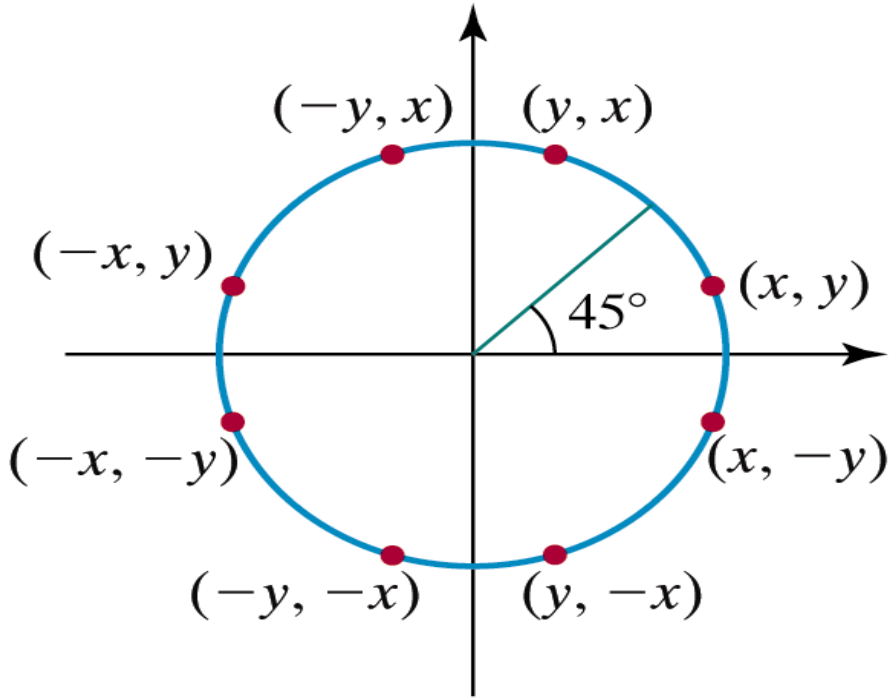


FIGURE 3-16 Circle with center coordinates (x_c, y_c) and radius r.

4.7 Circle-generating algorithms
2 Circle symmetry

- Symmetry simplifies circle scan – reduce computations.
- Need to scan only upper octant and scan all remaining circle *via* symmetry.



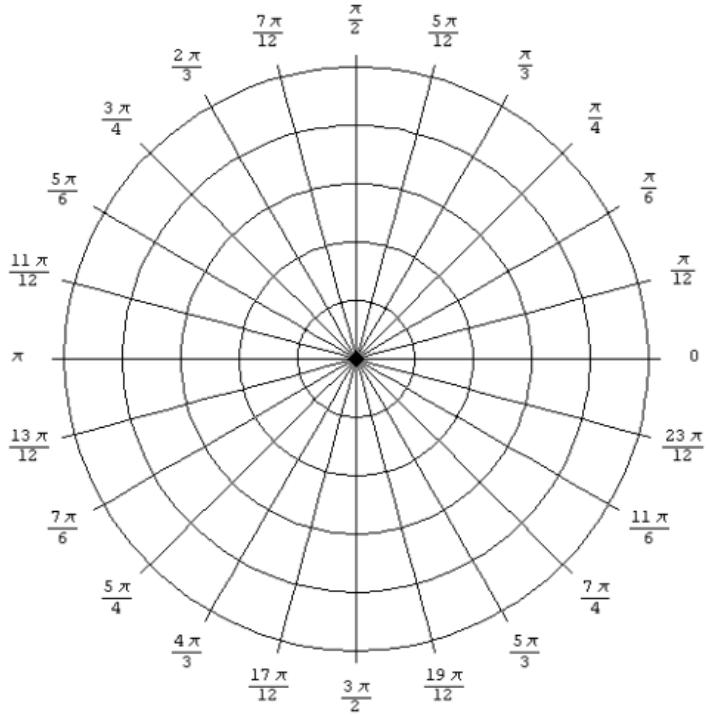


Figure 3-18

Symmetry of a circle. Calculation of a circle point (x, y) in one octant yields the circle points shown for the other seven octants.

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- Expressing the circle equation in parametric polar form yields the pair of equations. The polar and rectangular representation for a circle are:

$$x = r\cos\theta$$
$$y = r\sin\theta$$

$$y^2 + x^2 = r^2$$
$$y^2 = r^2 - x^2$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- Find a decision variable:

$$D(T) = (x_i+1)^2 + y_i^2 - r^2$$

$$D(S) = (x_i+1)^2 + (y_i - 1)^2 - r^2$$

Define decision variable (algebraic value)

$$d_i = D(S) + D(T)$$

$$d_i = (x_i + 1)^2 + y_i^2 - r^2 + (x_i + 1)^2 + (y_i - 1)^2 - r^2$$

$$d_i = 2(x_i + 1)^2 + (y_i - 1)^2 + y_i^2 - 2r^2$$

$$d_{i+1} = 2(x_{i+1} + 1)^2 + (y_{i+1} - 1)^2 + y_{i+1}^2 - 2r^2$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- Find the difference between decision variables to construct the recurrent relation:

$$d_{i+1} - d_i = 2(x_{i+1}+1)^2 + (y_{i+1}-1)^2 + y_{i+1}^2 - 2r^2 - 2(x_i+1)^2 - (y_i-1)^2 - y_i^2 + 2r^2$$

$$x_{i+1} = x_i + 1$$

$$d_{i+1} - d_i = 2(x_i+2)^2 + (y_{j+1}-1)^2 + (y_{j+1}^2) - 2(x_i+1)^2 - (y_i-1)^2 - y_i^2$$

$$= 2(x_i^2 + 4x_i + 4) + (y_{i+1}-1)^2 + y_{i+1}^2 - (y_i-1)^2 - y_i^2 - 2(x_i+1)^2$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- Find the difference between decision variables to construct the recurrent relation:

$$d_{i+1} - d_i = 2(x_i^2 + 4x_i + 4 - x_i^2 - 2x_i - 1) + (y_{i+1}^2 - 1)^2 + y_{i+1}^2 - (y_i - 1)^2 - y_i^2$$

$$d_{i+1} - d_i = 2(2x_i + 3) + (y_{i+1} - 1)^2 + y_{i+1}^2 - (y_i - 1)^2 - y_i^2$$

$$= 4x_i + 6 + y_{i+1}^2 + 1 - 2y_{i+1} + y_{i+1}^2 - (y_i^2 - 2y_i + 1) - y_i^2$$

$$= 4x_i + 6 + 2y_{i+1}^2 - 2y_{i+1} - y_i^2 + 2y_i - 1 - y_i^2 + 1$$

$$d_{i+1} - d_i = 4x_i + 2(y_{i+1}^2 - y_i^2) - 2(y_{i+1} - y_i) + 6$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- Here is the general recurrent relation:
- We have now to choose either pixel T or S

$$d_{i+1} = d_i + 4x_i + 6 + 2((y_i - 1)^2 - y_i^2) - 2((y_i - 1) - y_i)$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- Note that D(T)>0 (center of pixel T always outside the circle) and D(S)<0 (center of pixel S always inside the circle). See Slide 60.
- *Since* $d_i = D(T)+D(S)$
- *If* $d_i < 0$ ⇨ |D(T)|< |D(S)| ⇨ Chose pixel T since it is closer to circle, otherwise choose S.
- When $d_i < 0$:

$$y_{i+1} = y_i$$
$$d_{i+1} = d_i + 4x_i + 6$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

If $d_i >= o$ ⇨ choose S. In this case, we have:

$$y_{i+1} = y_i - 1$$

$$d_{i+1} = d_i + 4x_i + 6 + 2((y_i - 1)^2 - y_i^2) - 2((y_i - 1) - y_i)$$

$$= d_i + 4x_i + 6 + 2(y_i^2 - 2y_i + 1 - y_i^2) - 2(y_i - 1 - y_i)$$

$$d_{i+1} = d_i + 4(x_i - y_i) + 10$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- At last the final result is (nearly) found…
- We can now summarize the final equations…
- Note that the choice of T does not involve the y coordinate since the point remains on the same "height".

$$d_{i+1} = \begin{cases} d_i + 4x_i + 6, & d_i < 0, \quad \text{Choose T} \\ d_i + 4(x_i - y_i) + 10, & d_i \geq 0, \quad \text{Choose S} \end{cases} \qquad (25)$$

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.2 Mathematical derivation

- … but we need the initial value of the decision variable
- From the definition of d_i calculate d_o at the starting point (x_o=0, y_o=r) where r is the radius. We only need to scan the first octant starting from the y-axis and moving to the left until we reach the point where coordinates are equal (45° with x-axis)…

$$d_i = 2(x_i + 1)^2 + y_i^2 + (y_i - 1)^2 - 2r^2$$

$$d_0 = 2(0 + 1) + r^2 + (r - 1)^2 - 2r^2$$

$$d_0 = 3 - 2r$$

Exercise: Draw the first (top) octant for r=8.

4.7 Circle-generating algorithms
3 Bresenham's circle algorithm
3.3 Pseudo code

- 1. input radius r and circle center $(x_c.y_c)$ then choose first point to scan $(x_0,y_0)=(0,r)$.
- 2. calculate initial decision variable $d_0=3-2r$
- 3. At each point, starting at k=0, perform the following test:
- 3.1 if $d_K<0$ then the next point along the circle centered on (0,0) is (x_{K+1}, y_K) and $d_{k+1}=d_k+4x_k+6$
- 3.2 otherwise the next point along the circle is (x_{K+1}, y_K-1) with $x_{k+1}=x_k+1$ and $y_{k+1}=y_k-1$ and $d_{k+1}=d_k+4(x_k-y_k)+10$
- 4. Determine symmetry points in other 7 octants
- 5. Move each calculated pixel position (x,y) onto the circular path centered at (x_c, y_c), and plot coordinate values: $x=x+x_c$, $y=y+y_c$
- 6. Repeat 3 to 5 until x>=y (end of first (top) octant).

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.4 OpenGL Code (1 of 5) (adapt it!)

```
#include <GLUT/GLUT.h>
 class screenPoint
{ private:    GLint x,y;
   public:
   /* Defult constructor: initializes coordinates position to (0,0) */
   screenPoint()
   {       x = 0;       y = 0;    }
     //Methods
   void setCoordinates(GLint xCoordinateValue, GLint yCoordinateValue)
   {       x = xCoordinateValue;       y = yCoordinateValue;    }
      GLint getx () const      {       return x;     }
      GLint gety () const      {       return y;     }
      void incrementx ()     {       x++;    }
   void decrementy ()      {       y--;    }
   };
```

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.4 OpenGL Code (2 of 5) (adapt it!)

```
void setPixel(GLint xCoordinate, GLint yCoordinate)
{  glBegin(GL_POINTS);
   glVertex2i(xCoordinate, yCoordinate);
   glEnd();
   glFlush(); //process all OpenGL functions as quickly as possible
}
 /*void circleMidPoint(GLint xc, GLint yc, GLint raduis)
{    screenPoint circlePoint;  //initialize value for midpoint parameter
   GLint p = 1-raduis;      //set coordinates for top point of circle
   circlePoint.setCoordinates(0,raduis);     //function prototype
   void circlePlotPoints(GLint, GLint, screenPoint); //plot the initial point in
each quadrant
   circlePlotPoints(xc, yc, circlePoint);
      //calculate the next point and plot in each octant
   while(circlePoint.getx() < circlePoint.gety())
   {   circlePoint.incrementx();
      if(p<0)              p += 2 * circlePoint.getx() + 3;
      else   {   circlePoint.decrementy ();
         p += 2 * (circlePoint.getx () - circlePoint.gety ()) + 5;        }
      circlePlotPoints(xc, yc, circlePoint);
   } } */
```

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.4 OpenGL Code (3 of 5) (adapt it!)

```
void Brscircle (GLint xc, GLint yc, GLint raduis)
{   screenPoint circlePoint;    //initialize value for midpoint parameter
   GLint p = 3-2*raduis;    //set coordinates for top point of circle
   circlePoint.setCoordinates(0,raduis);       //function prototype
   void circlePlotPoints(GLint, GLint, screenPoint);
     //plot the initial point in each quadrant
   circlePlotPoints(xc, yc, circlePoint);
     //calculate the next point and plot in each octant
   while(circlePoint.getx() < circlePoint.gety())
   {       circlePoint.incrementx();
     if(p<0)          p += 4 * circlePoint.getx() + 6;
     else{          circlePoint.decrementy ();
        p += 4 * (circlePoint.getx () - circlePoint.gety ()) + 10;       }
     circlePlotPoints(xc, yc, circlePoint);
   }   }
```

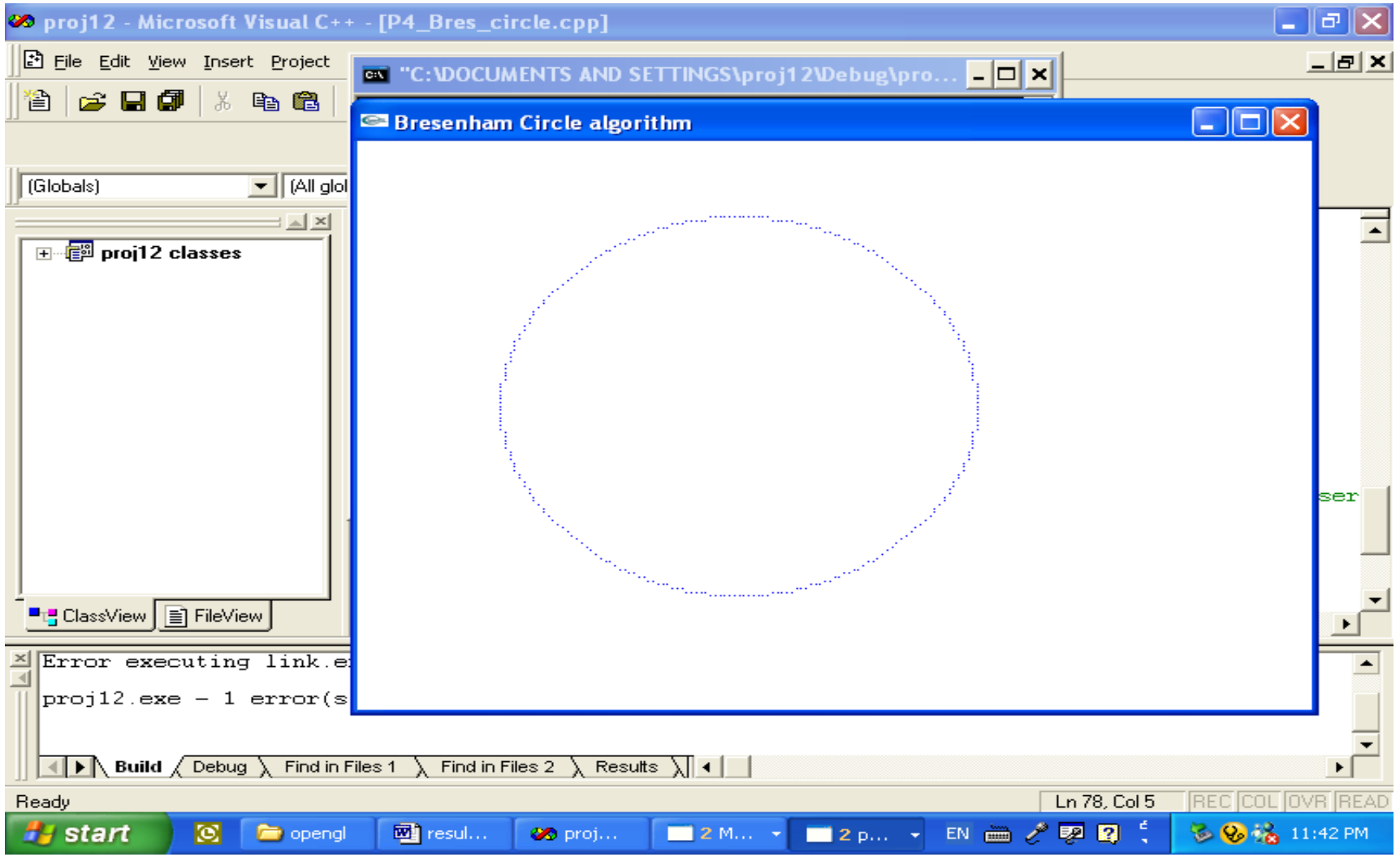
4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.4 OpenGL Code (4 of 5) (adapt it!)

```
void circlePlotPoints(GLint xc, GLint yc, screenPoint circPoint)
{   setPixel(xc + circPoint.getx () , yc + circPoint.gety ());
   setPixel(xc - circPoint.getx () , yc + circPoint.gety ());
   setPixel(xc + circPoint.getx () , yc - circPoint.gety ());
   setPixel(xc - circPoint.getx () , yc - circPoint.gety ());
   setPixel(xc + circPoint.gety () , yc + circPoint.getx ());
   setPixel(xc - circPoint.gety () , yc + circPoint.getx ());
   setPixel(xc + circPoint.gety () , yc - circPoint.getx ());
   setPixel(xc - circPoint.gety () , yc - circPoint.getx ());   }
void init(void) //Initialize OpenGL
{  glClearColor(1,10.0,10.0,0.0); //set display-window color to black
   glMatrixMode(GL_PROJECTION); //set projection parameters
   gluOrtho2D(0.0,300.0,0.0,300.0);  }
```

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.4 OpenGL Code (5 of 5) (adapt it!)

```
void drawMyCircle(void)
{   glClear(GL_COLOR_BUFFER_BIT); //clear display-window
   glColor3f(0,255,255); //set pixel color to red
   glPointSize(3.0); int xCenter = 150; int yCenter = 150; int raduis = 100;
   Brscircle(xCenter, yCenter, raduis);   }
 int main(int argc, char**argv)
{     glutInit(&argc, argv);  //initialize GLUT
   glutInitWindowPosition(10,10); //set top-left display-window position
   glutInitWindowSize(500,500); //set display-window width and height to 500
   glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  //set display mode
      //Now display the window with a title
   glutCreateWindow("Circle Bresenham & Mid Point Algorithm - Program");
      init(); //execute initialization procedure of OpenGL
   glutDisplayFunc(drawMyCircle); //send graphics to display window
   glutMainLoop(); //display everything and wait
    return 0;  }
```

4.7 Circle-generating algorithms
3 Bresenham's Circle algorithm
3.5 OpenGL code test



4.7 Circle-generating algorithms
4.8 MidPoint Circle algorithm

- Here is the idea. (Read pp.105-109 Text)
- Use your experience acquired in deriving Bresenham's algorithm to mathematically derive the Midpoint circle algorithm.
- Write the pseudo code and openGL code and test it.

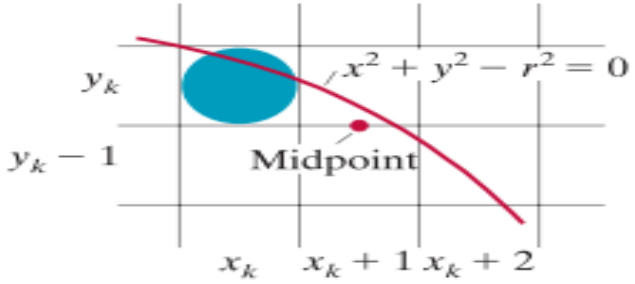
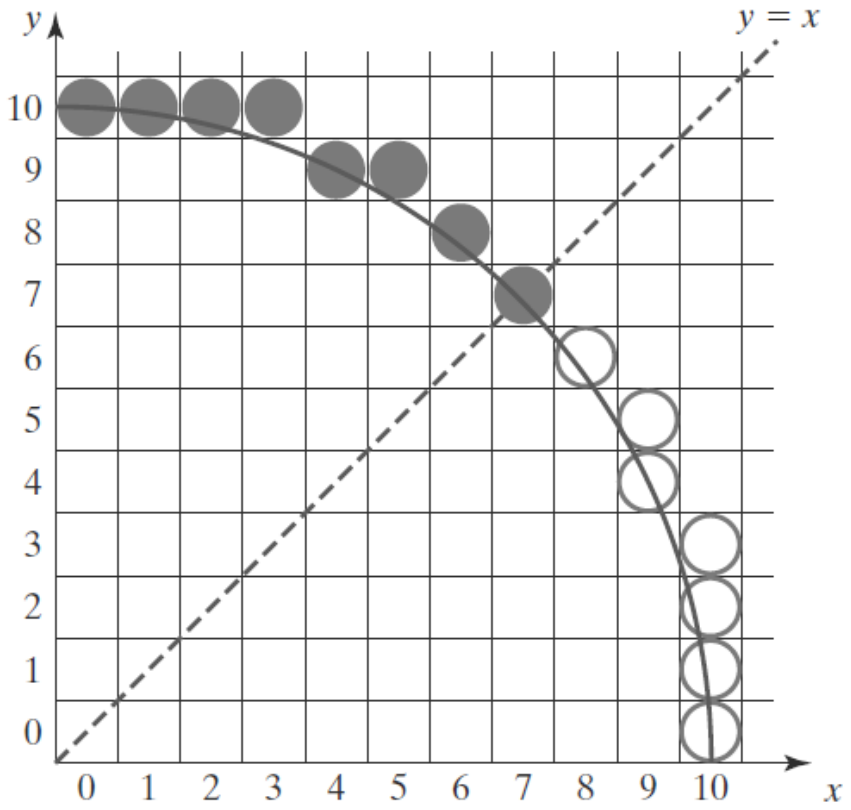
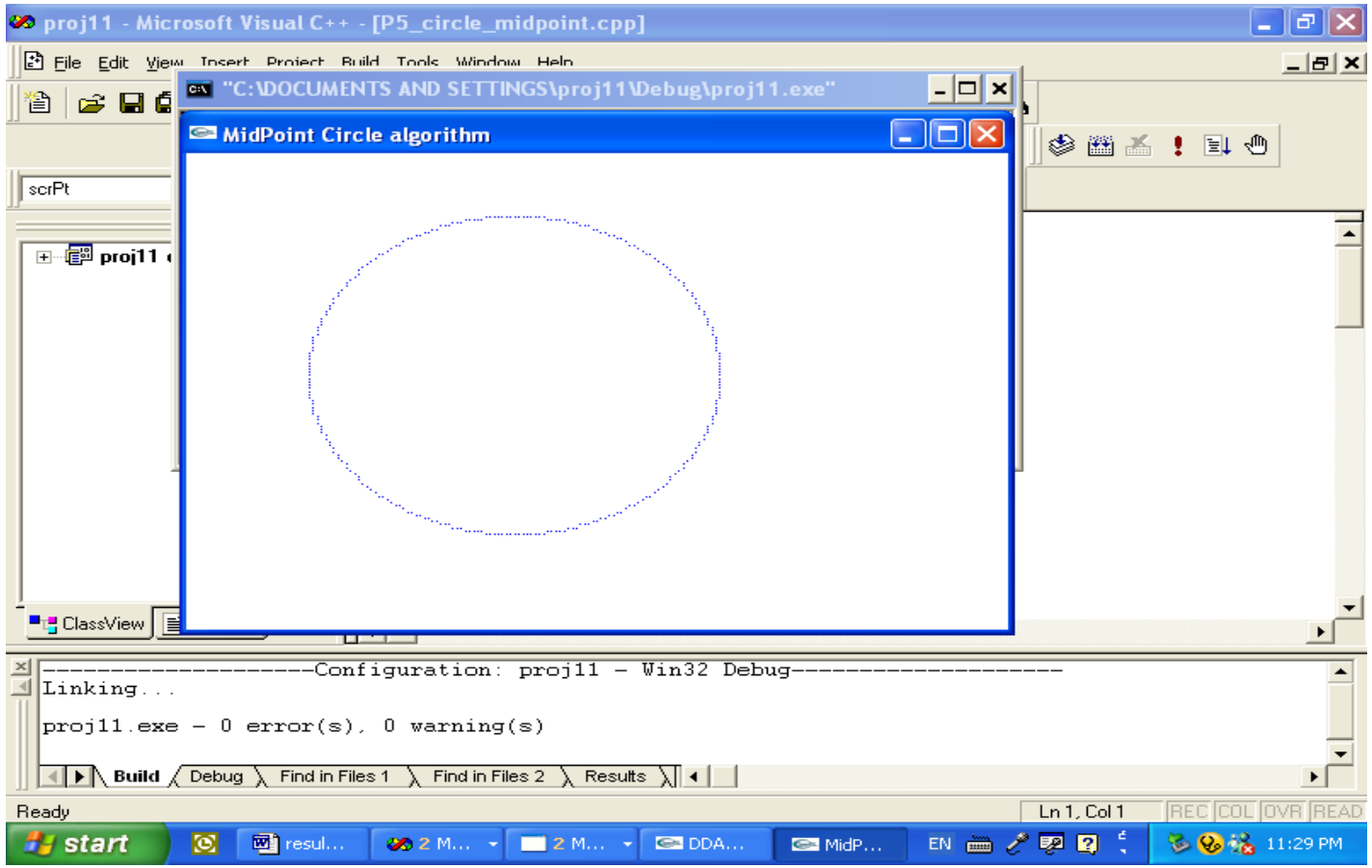


FIGURE 3–19 Midpoint between candidate pixels at sampling position $x_k + 1$ along a circular path.



4.7 Circle-generating algorithms
4. MidPoint OpenGL code test



4.8 Ellipse-generating algorithms
5. MidPoint algorithm

- **Exercise:**

Similarly to line drawings and circle drawings, you have to follow the complete treatment for the ellipse from ideas to program testing through complete mathematical derivation (See pp. 109-116)

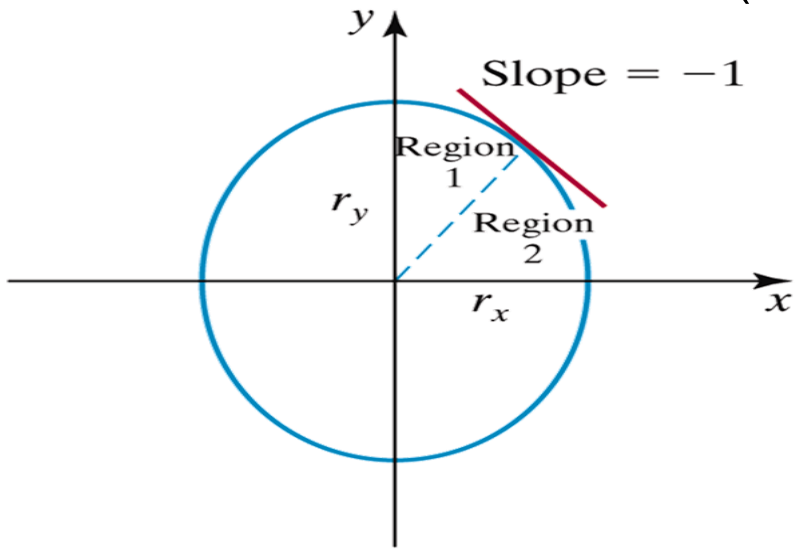


Figure 3-25

Ellipse processing regions. Over region 1, the magnitude of the ellipse slope is less than 1.0; over region 2, the magnitude of the slope is greater than 1.0.

4.9 Polygon fill-areas

1. inside-outside test

- See page 127-129
- Odd-parity rule

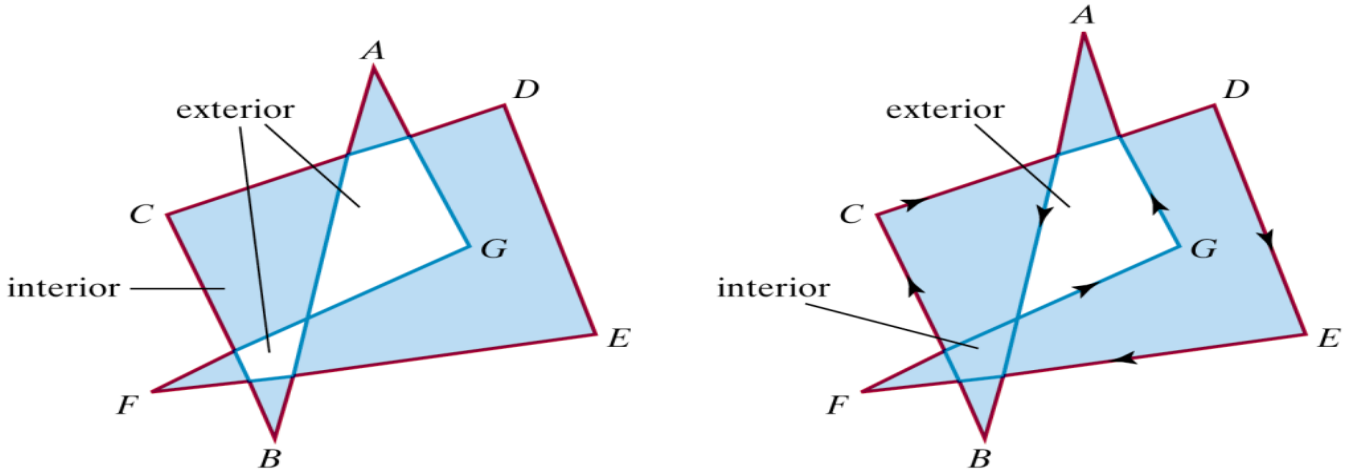
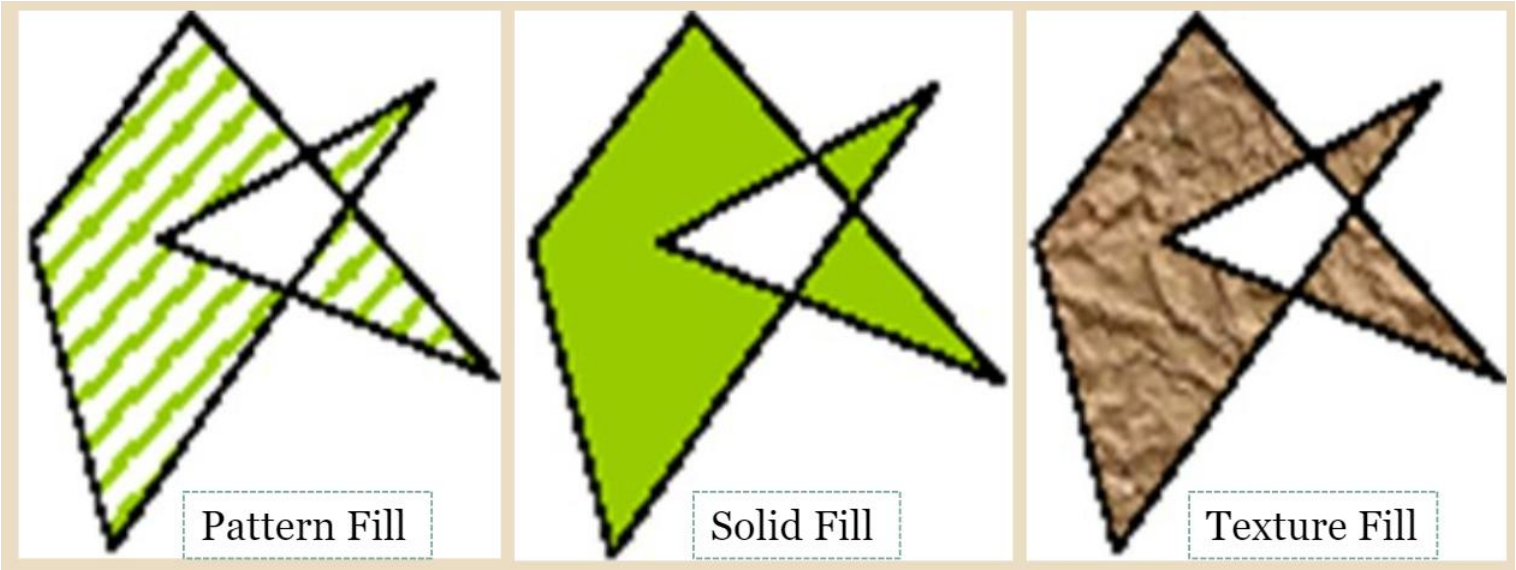


Figure 3-46

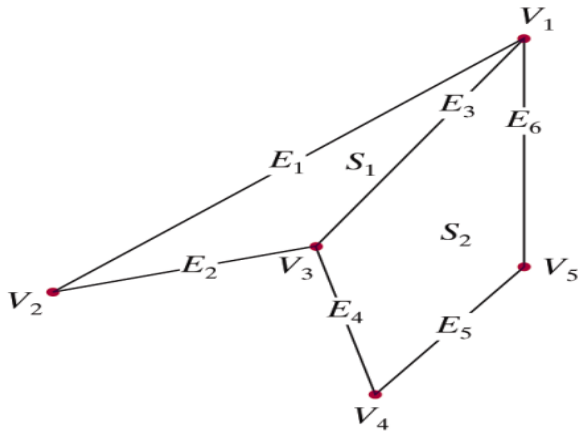
Identifying interior and exterior regions of a closed polyline that contains self-intersecting segments.

Types of Polygon fill-areas



4.9 Polygon fill-areas
2. Polygon table

- See page 129-131, Odd-parity rule



VERTEX TABLE	
V_1:	x_1, y_1, z_1
V_2:	x_2, y_2, z_2
V_3:	x_3, y_3, z_3
V_4:	x_4, y_4, z_4
V_5:	x_5, y_5, z_5

EDGE TABLE	
E_1:	V_1, V_2
E_2:	V_2, V_3
E_3:	V_3, V_1
E_4:	V_3, V_4
E_5:	V_4, V_5
E_6:	V_5, V_1

SURFACE-FACET TABLE	
S_1:	E_1, E_2, E_3
S_2:	E_3, E_4, E_5, E_6

Figure 3-50

Geometric data-table representation for two adjacent polygon surface facets, formed with six edges and five vertices.

4.9 Polygon fill-areas
3. Front and back polygon faces

- See page 132-133

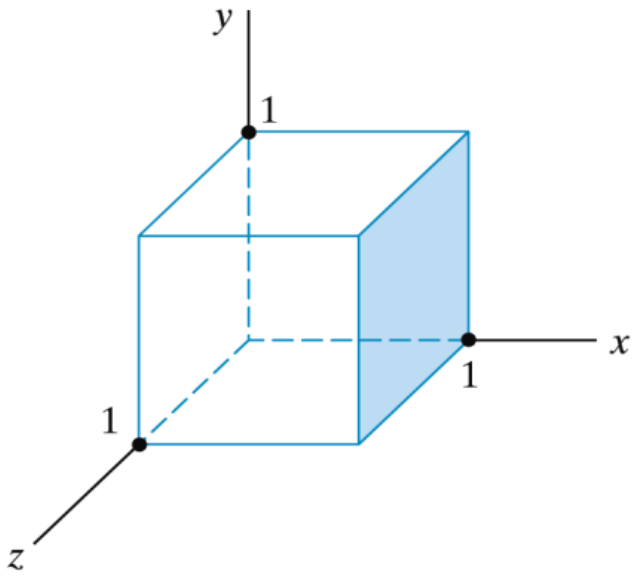


Figure 3-52

The shaded polygon surface of the unit cube has plane equation x – 1 = 0.

4.9 Polygon fill-areas
5. OpenGL POLYGON FILL-AREA FUNCTIONS

- See pages 134 – 139
- Write OpenGL programs to display fill-polygon and test them in LAB.

- Go through TABLE 3-1 – Summary of OpenGL Output Primitive Functions and Related routines. See pages 158 & 159

Summary & Homework

- Draw individual points, lines, circles, ellipses, and other curves
- 3 methods for locating a pixel on a straight-line: Direct Differential Analyzer, Bresenham's algorithm, midpoint method.
- Circles and ellipses can be efficiently be plotted using Bresenham's algorithm, midpoint method.
- Method we follow considers going from ideas to mathematics, to pseudo code to code writing and testing for all algorithms of concern.
- Fill-area methods display in a solid color or color pattern.

Assignment 2

Based on the DDA algorithm:

- Calculate the points between the starting point (5, 6) and ending point (13, 10). **??**
- Then, draw the line using these points?
- ***SUBMIT through the BLACKBOARD……***

Thank you